(Deep) Convolutional Neural Networks Part I

Recap: Deep Learning

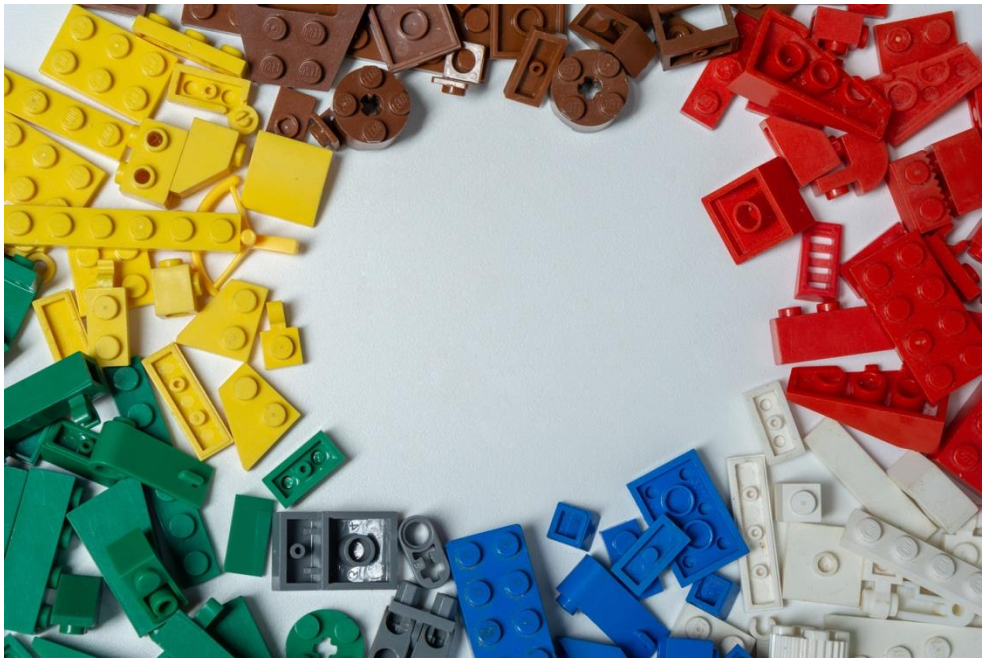
- (Deep) Neural Network:
  Composing a set of (nonlinear) functions g

$$f(\boldsymbol{x};\boldsymbol{\theta}) = g_1(\dots g_{n-1}(g_n(\boldsymbol{x};\boldsymbol{\theta_n}),\boldsymbol{\theta_{n-1}})\dots,\boldsymbol{\theta_1})$$

- Forward propagation: compute the output given the input and the parameters
- Backward propagation: compute the gradients of the parameters by comparing the output to a target label using a loss function
- Training: Finding parameters that minimize the average loss on the training set

Deep learning: Key Questions

How to design individual operations?



How to compose these functions?



Deep learning: Key Questions

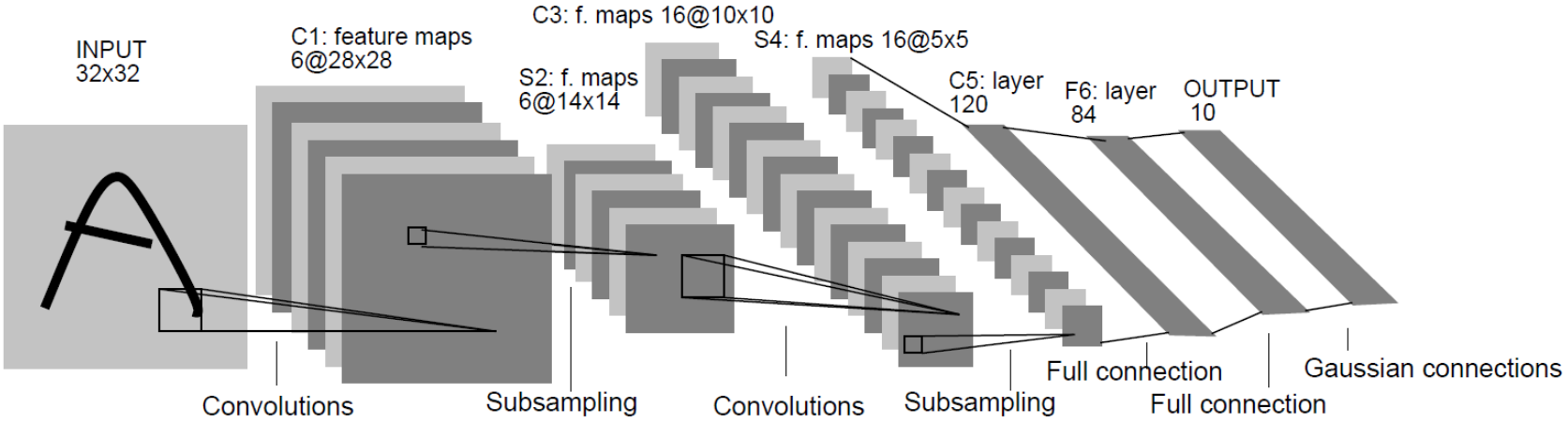
How to design individual operations?

- Fully connected
- Convolution
- Normalization
- Nonlinear activation
- Pooling
- …

How to compose these functions?

- Convolutional neural networks
  - LeNet
  - AlexNet
  - GoogLeNet
  - ResNet
  - DenseNet
  - …

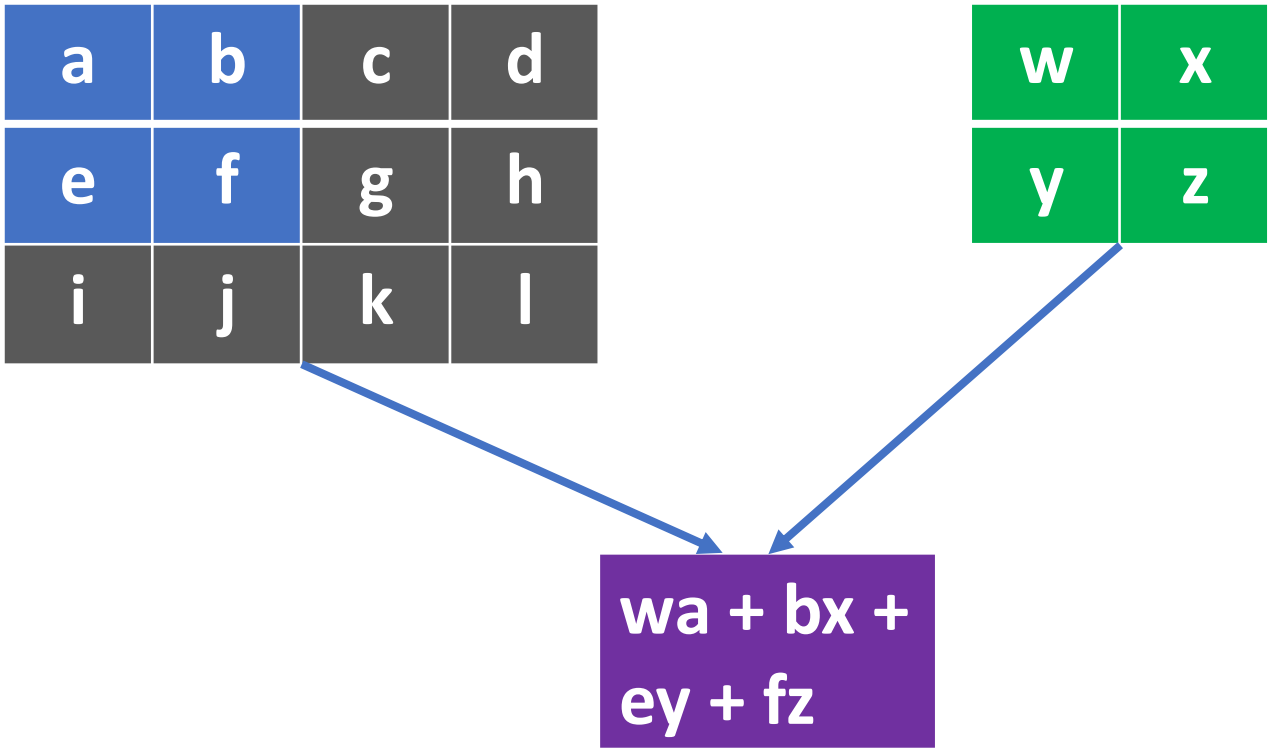
A Convolutional Network (LeNet) in 2022



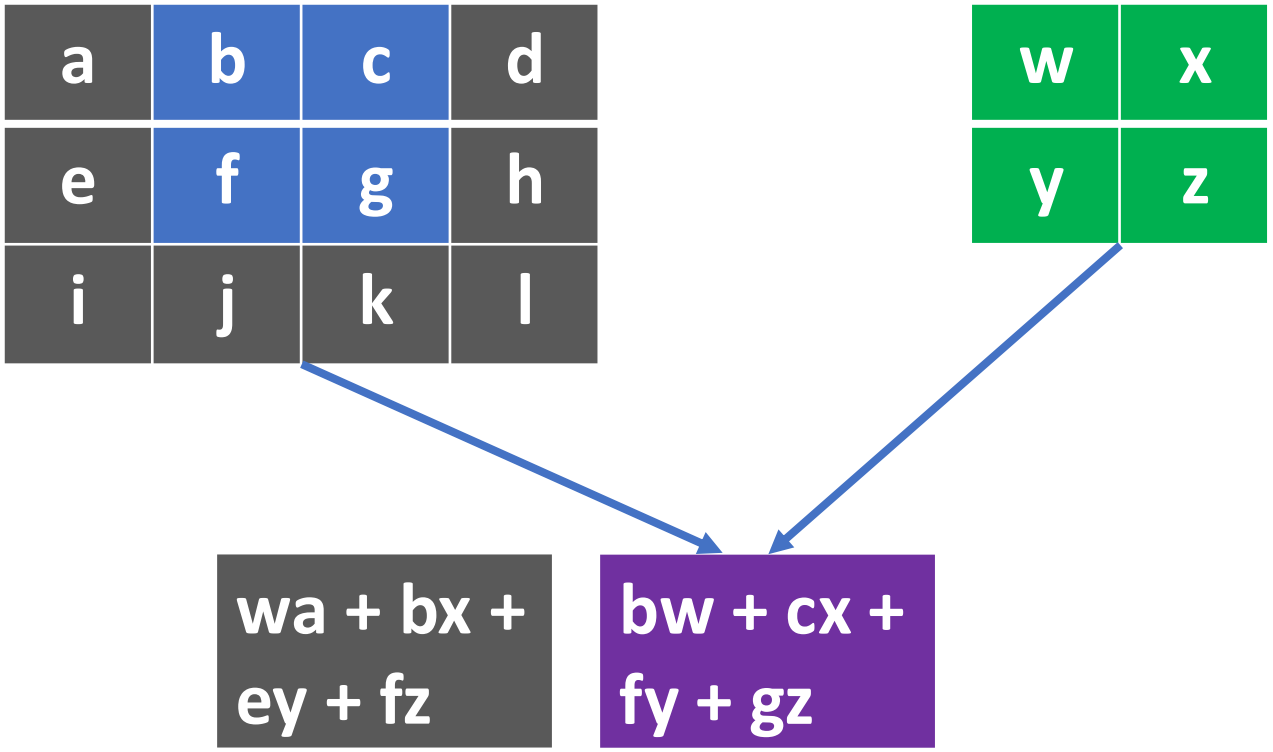
[LeCun et al., 1998]

Convolution

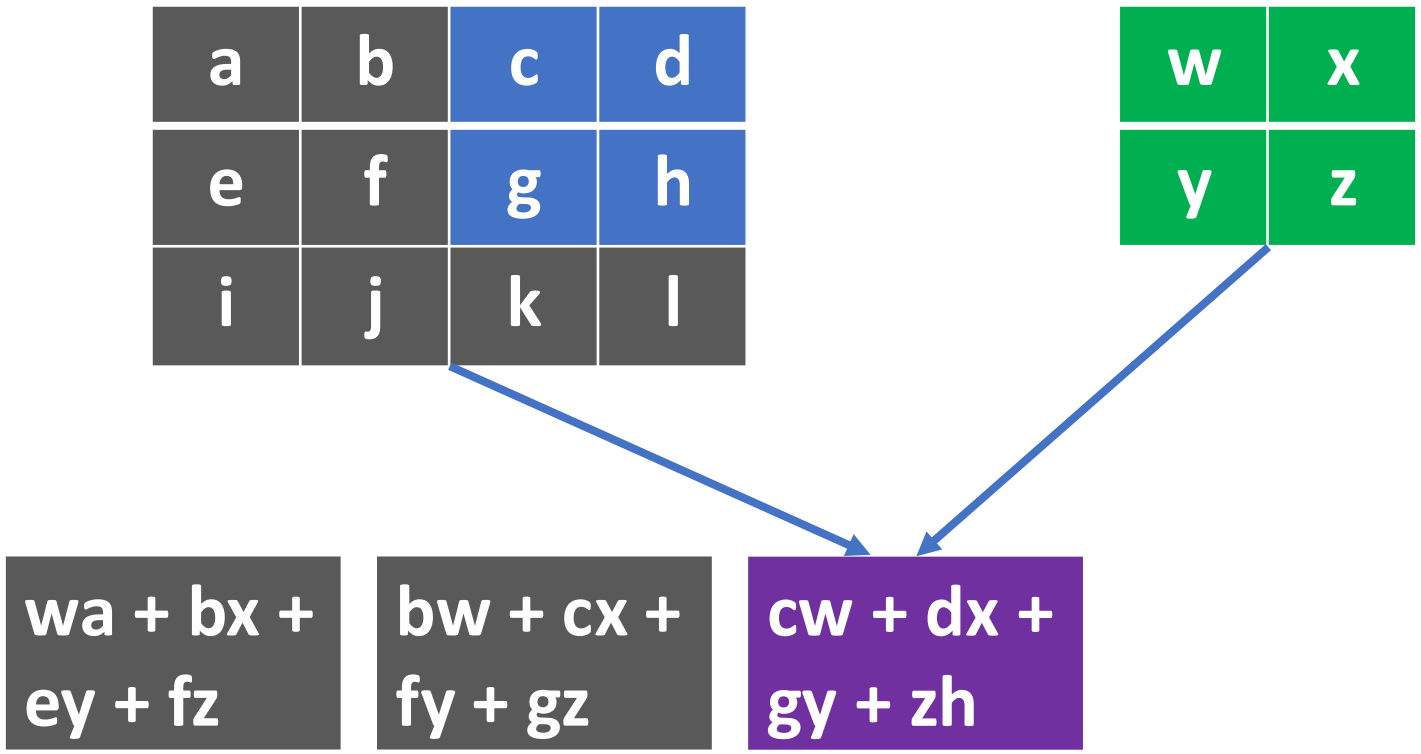
Convolution: An Example



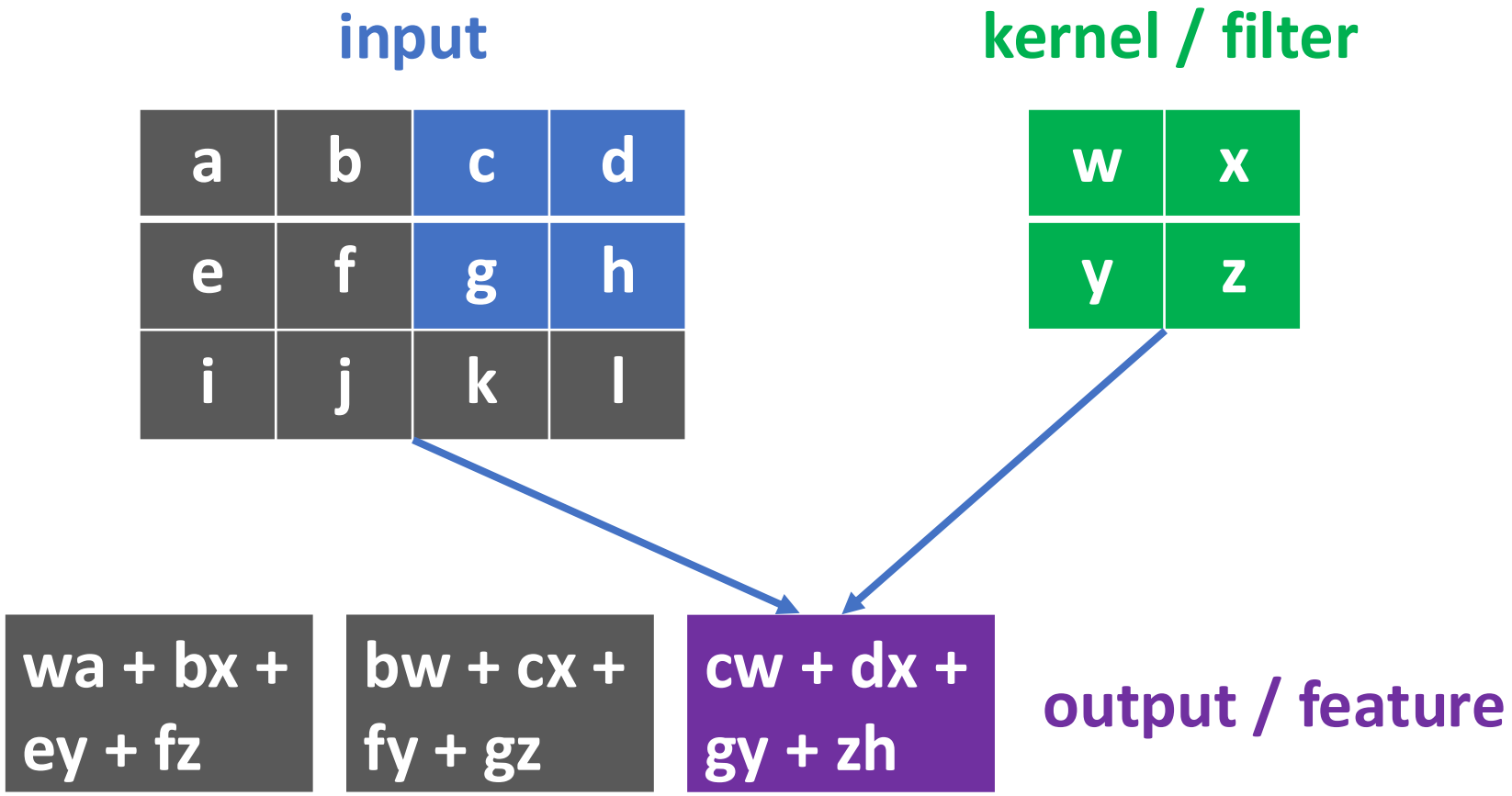
Convolution: An Example



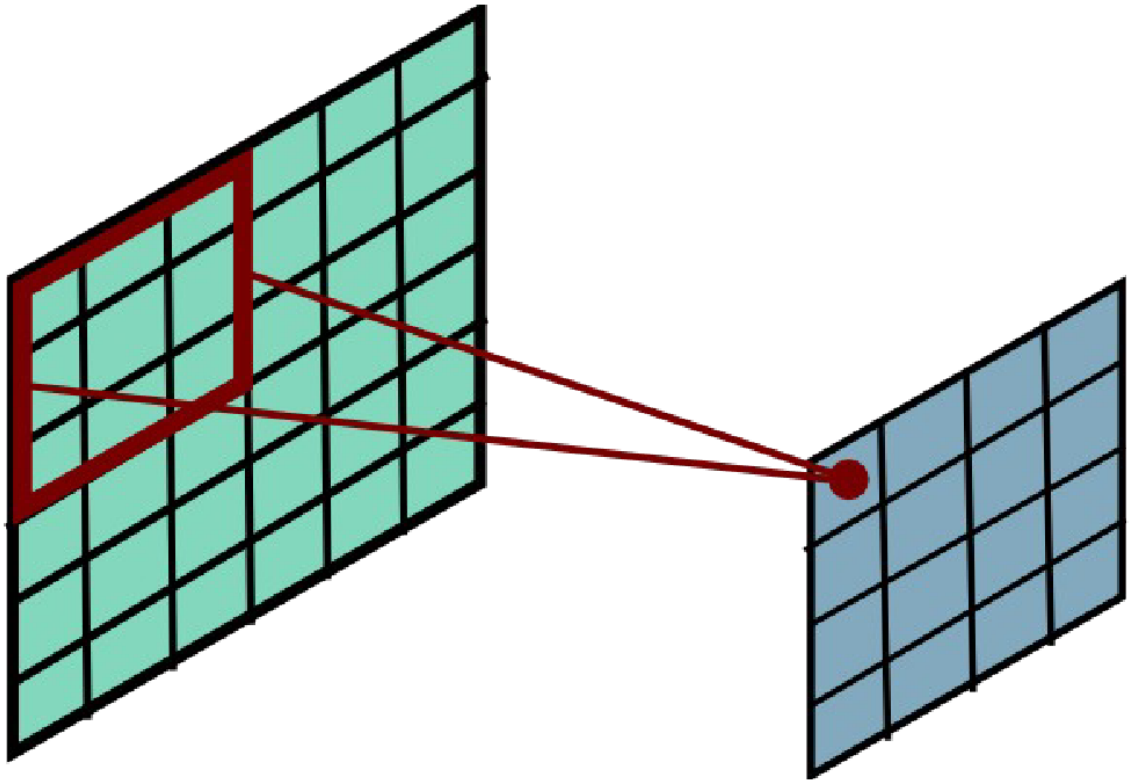
Convolution: An Example

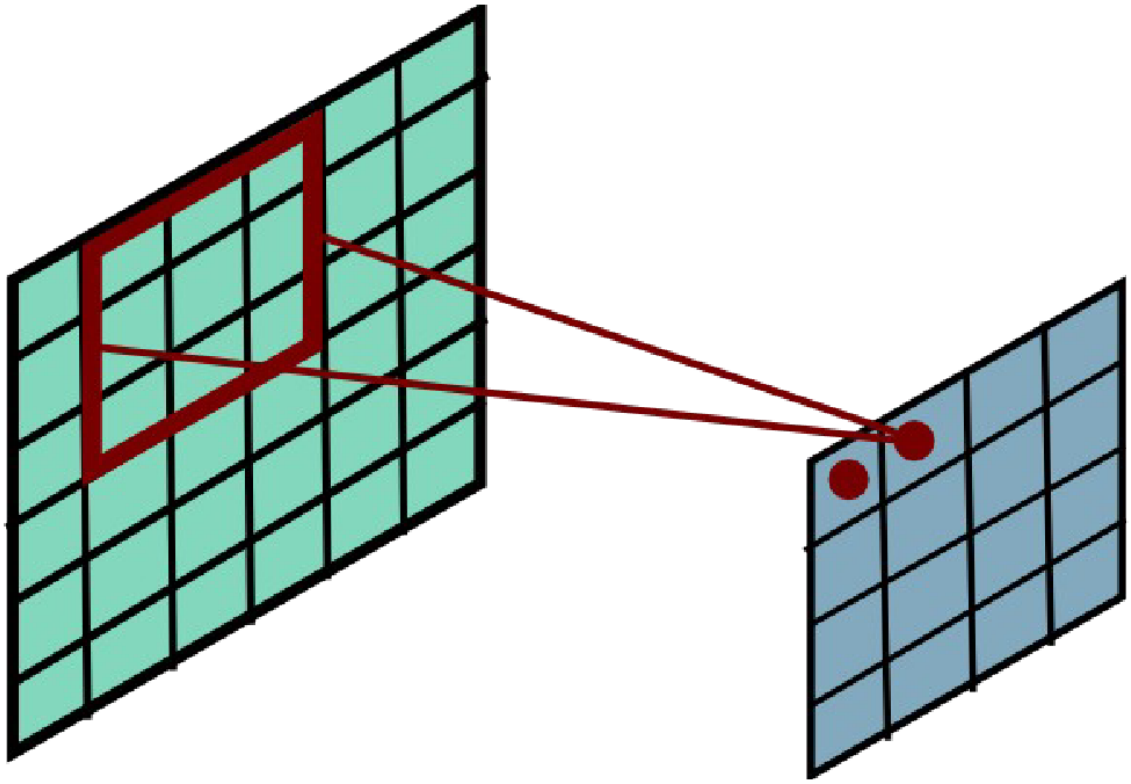


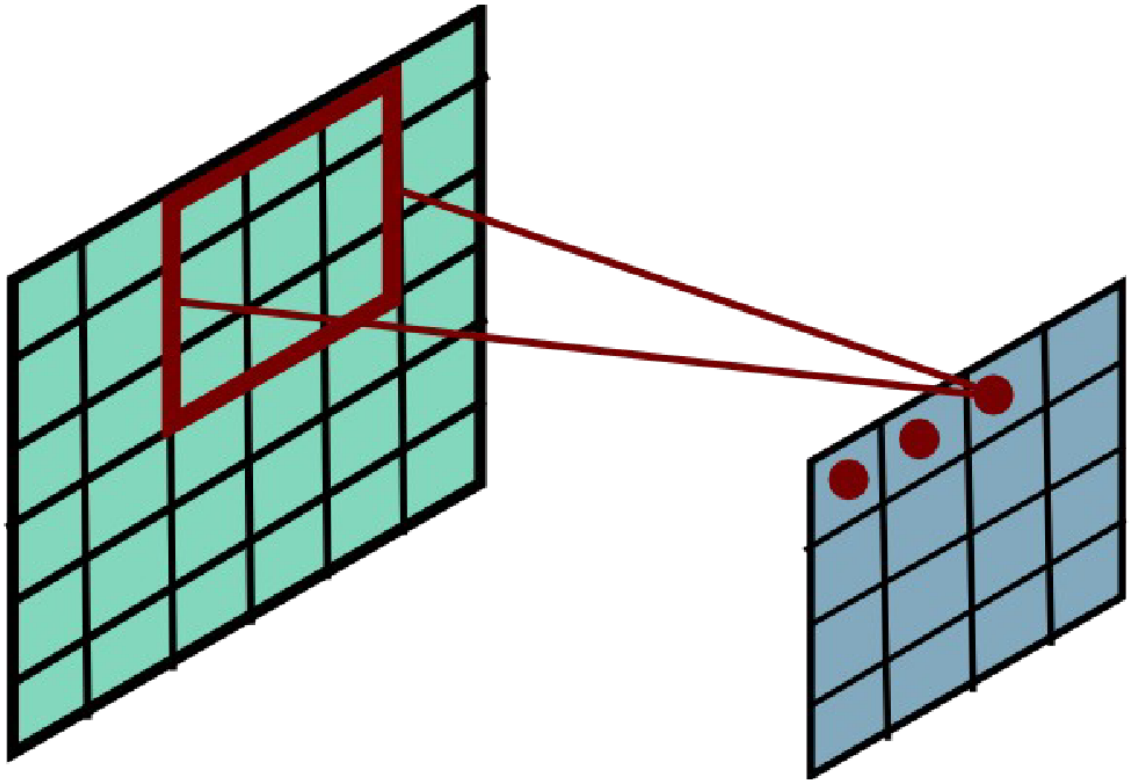
Convolution: An Example

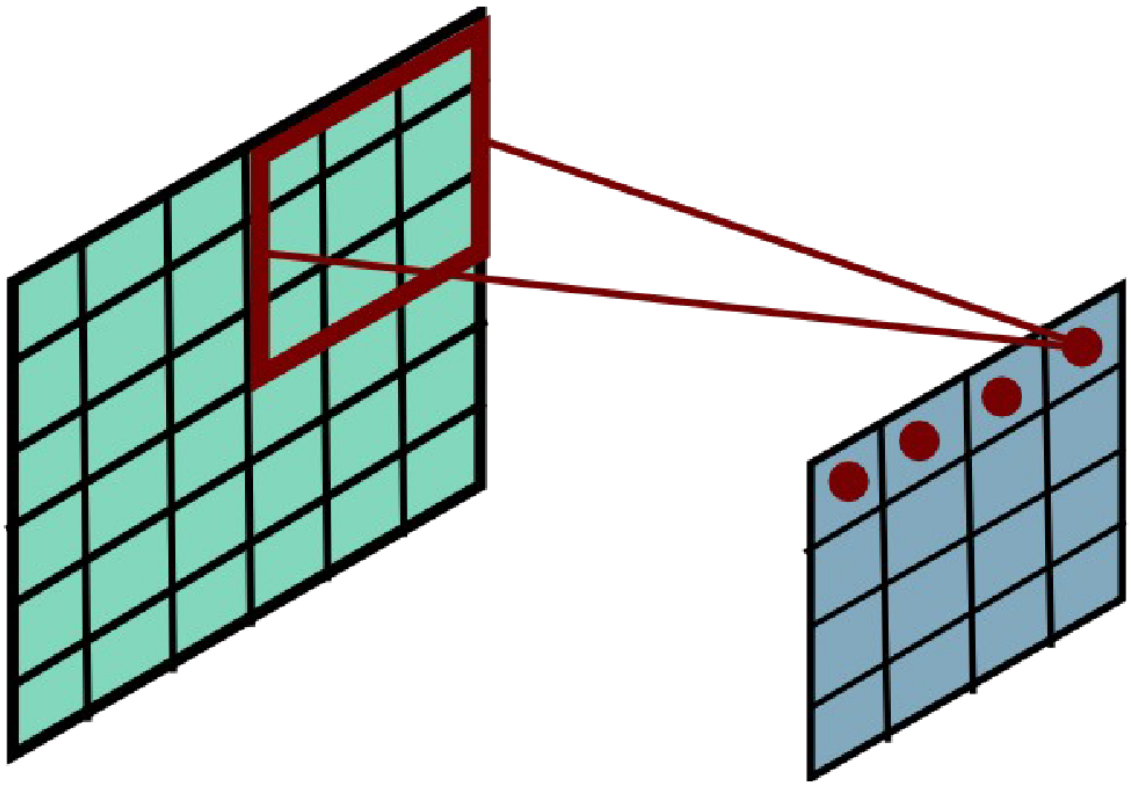


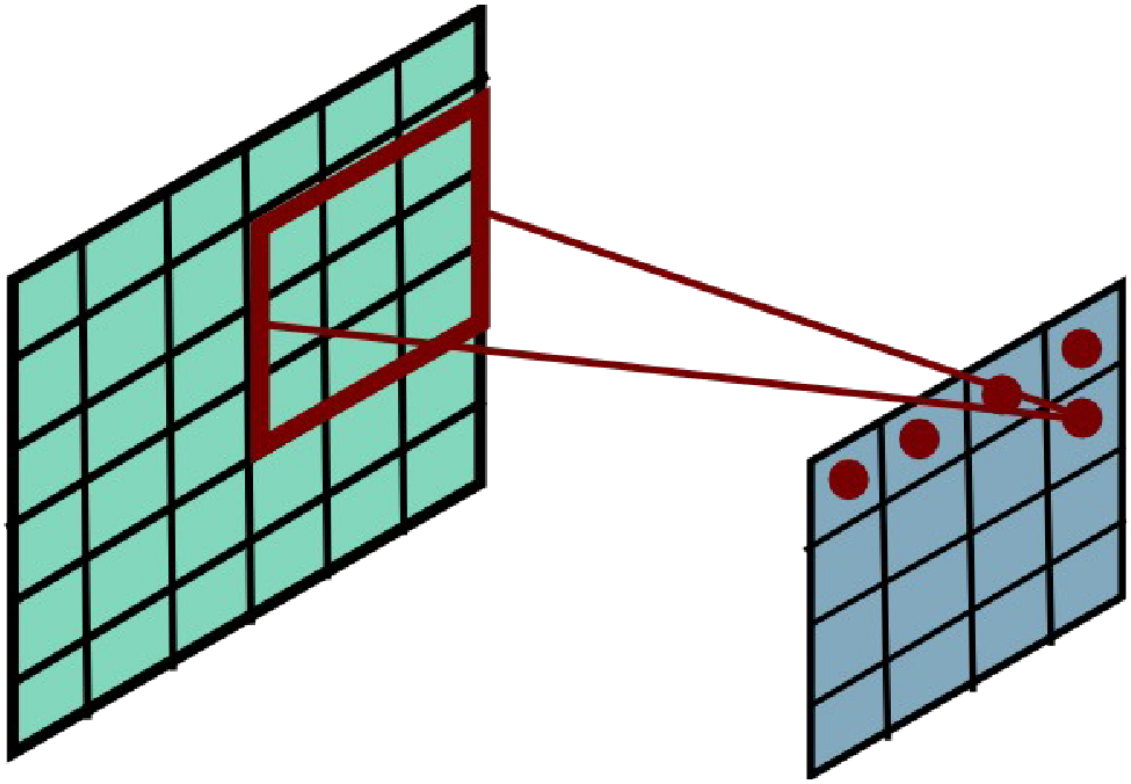
* This is technically **correlation**, yet we don't care about the difference between correlation and convolution here

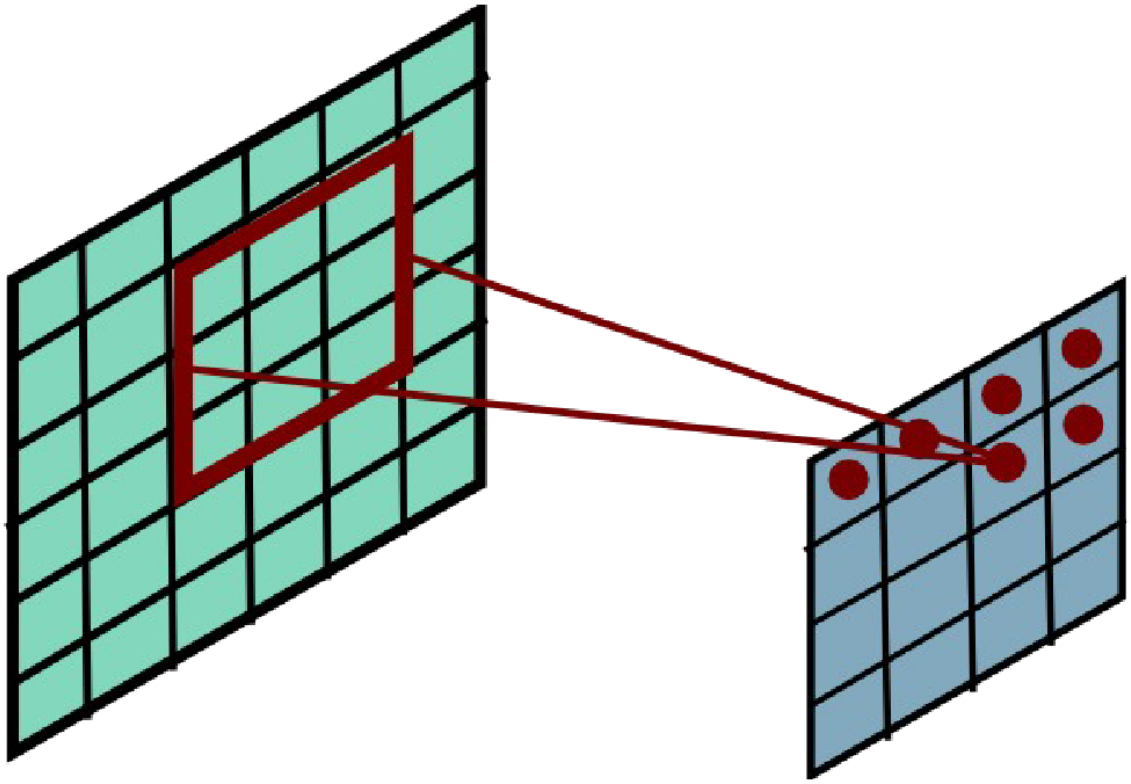


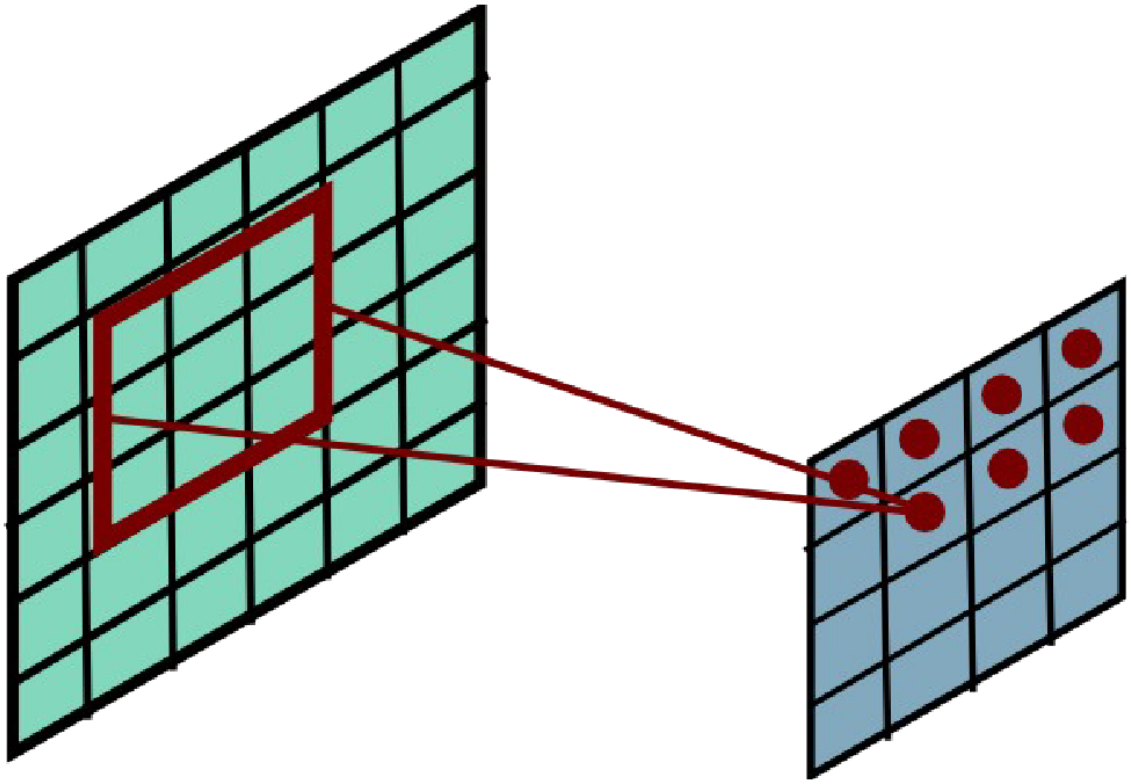


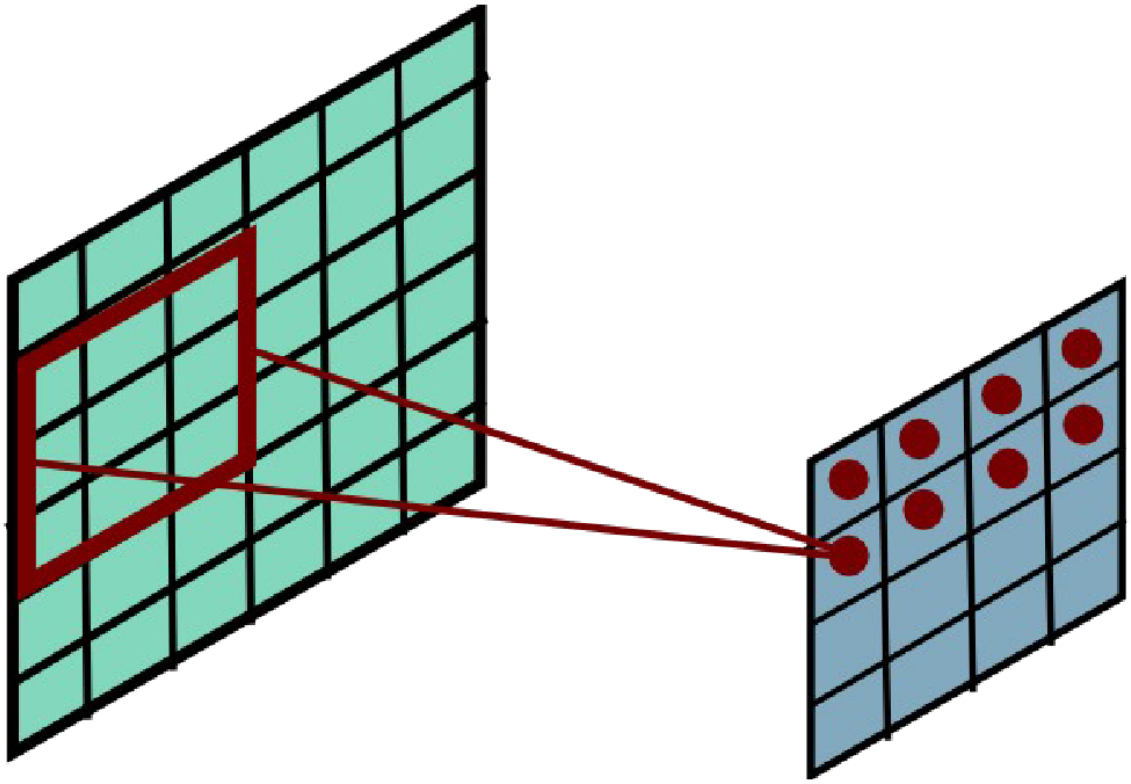


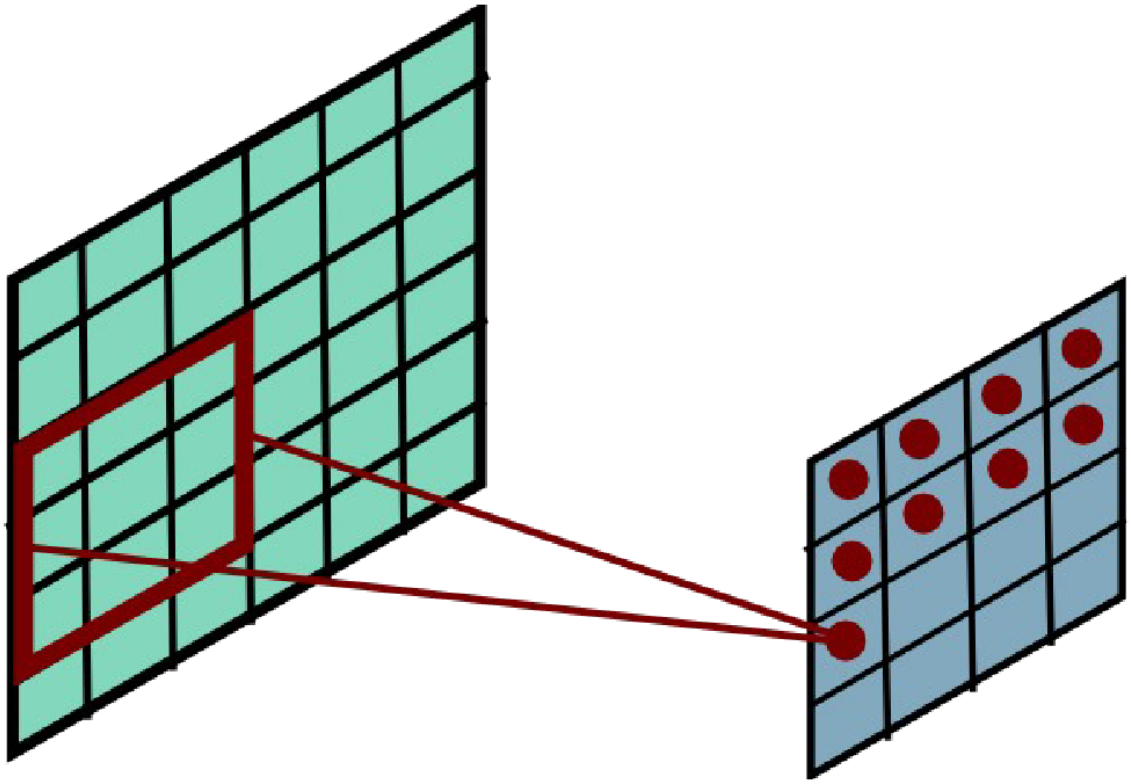


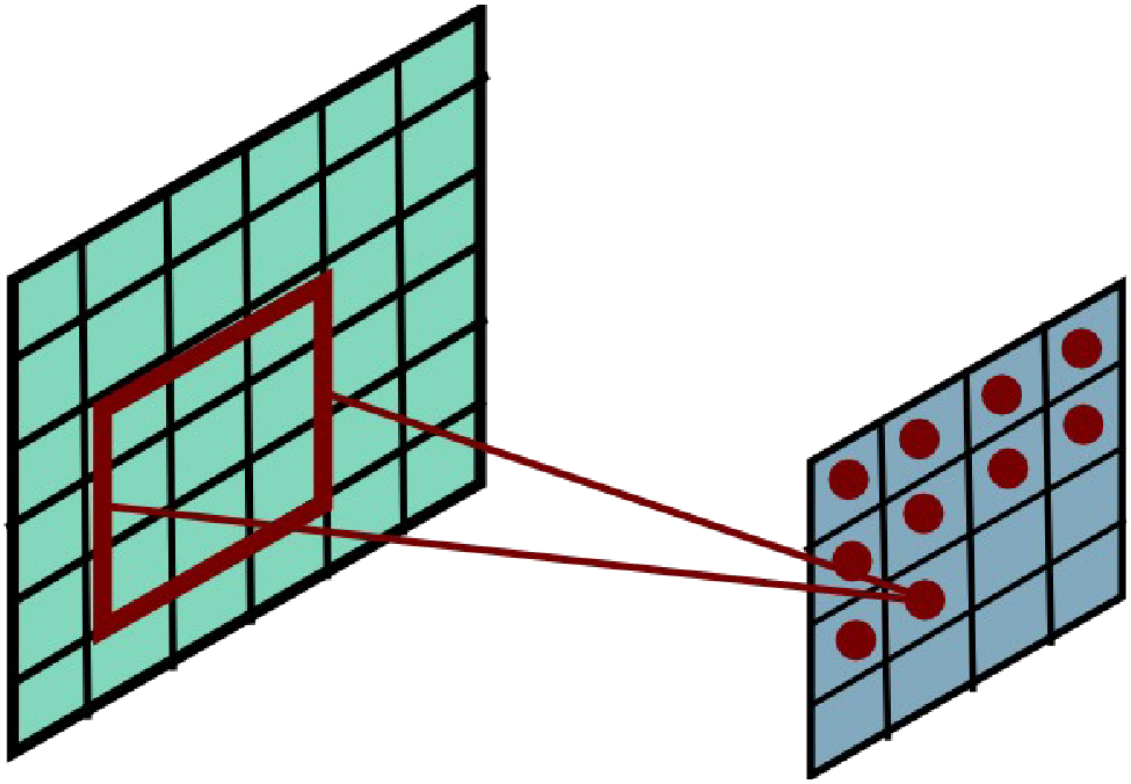


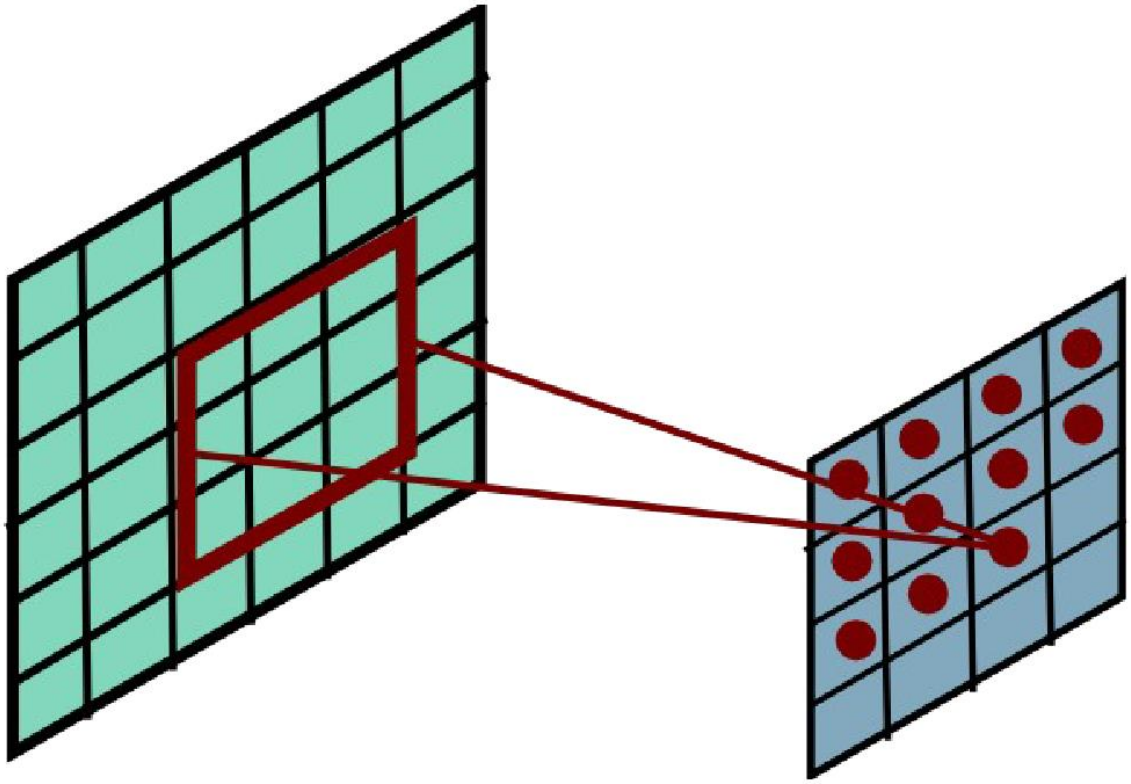


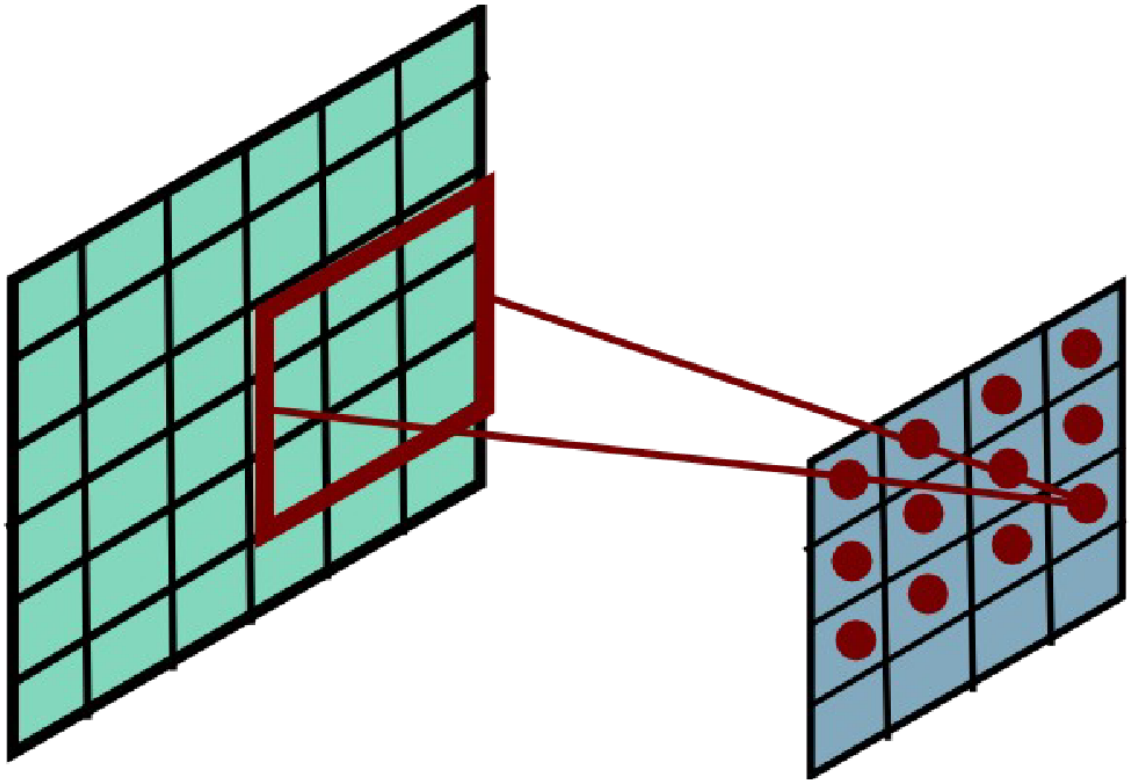


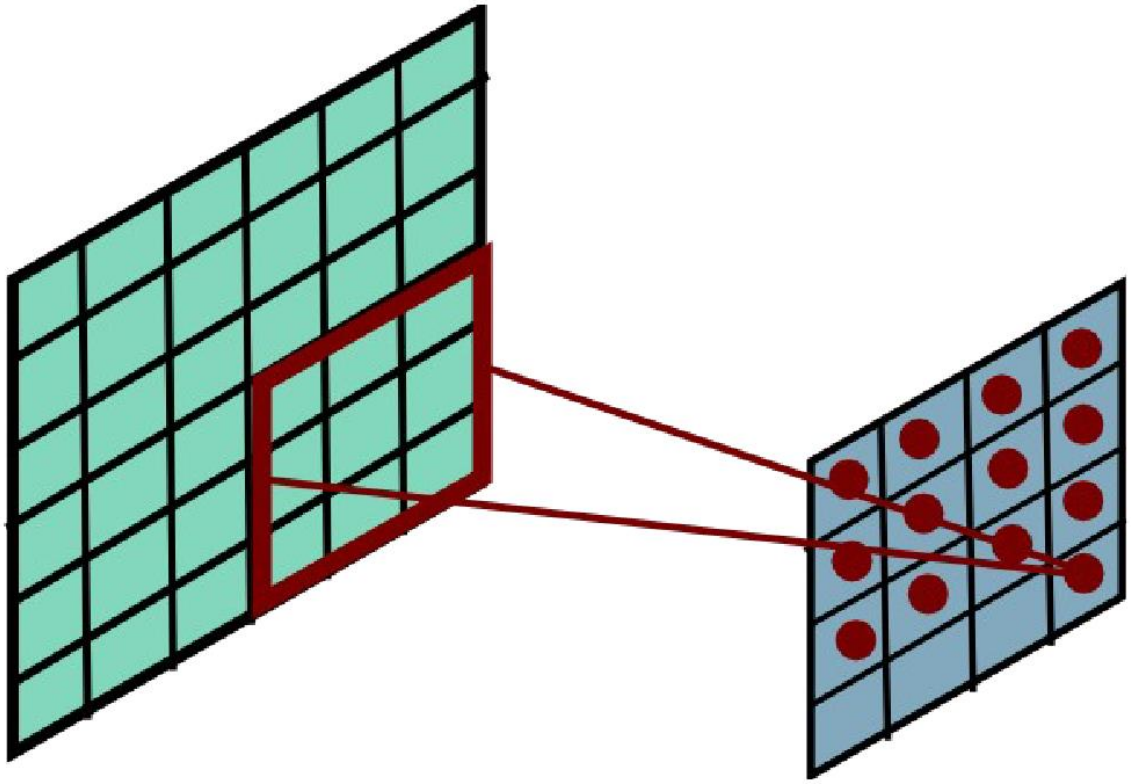


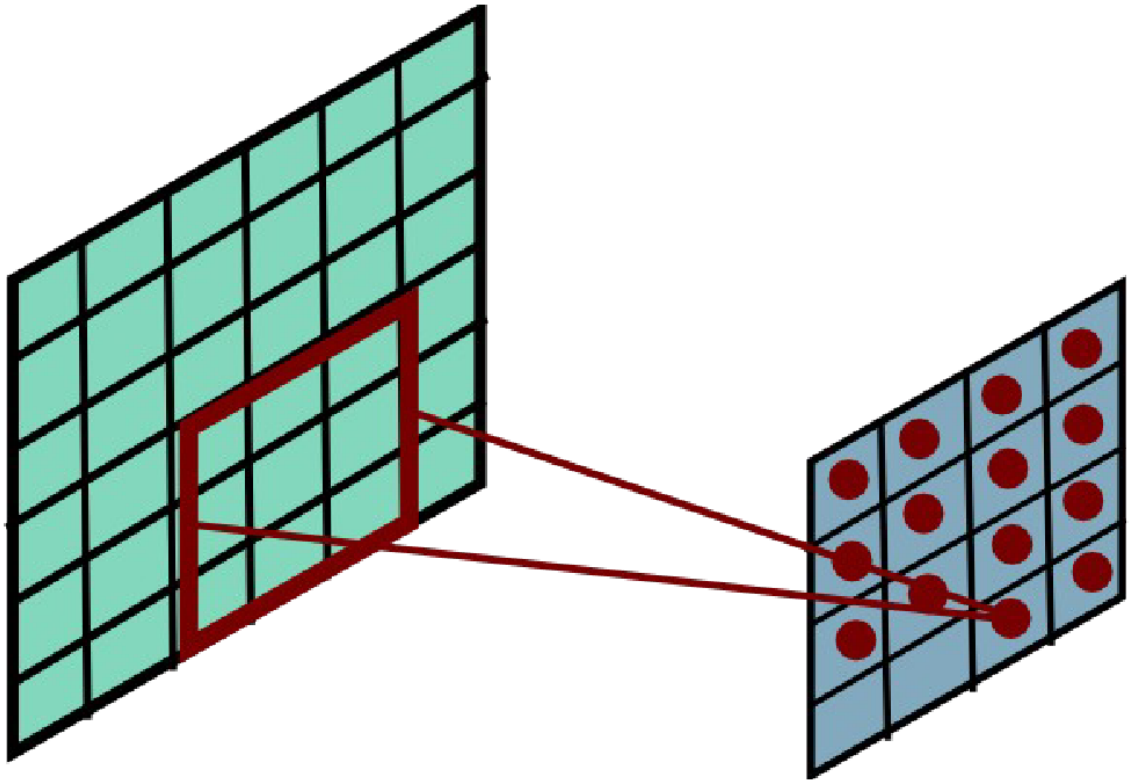


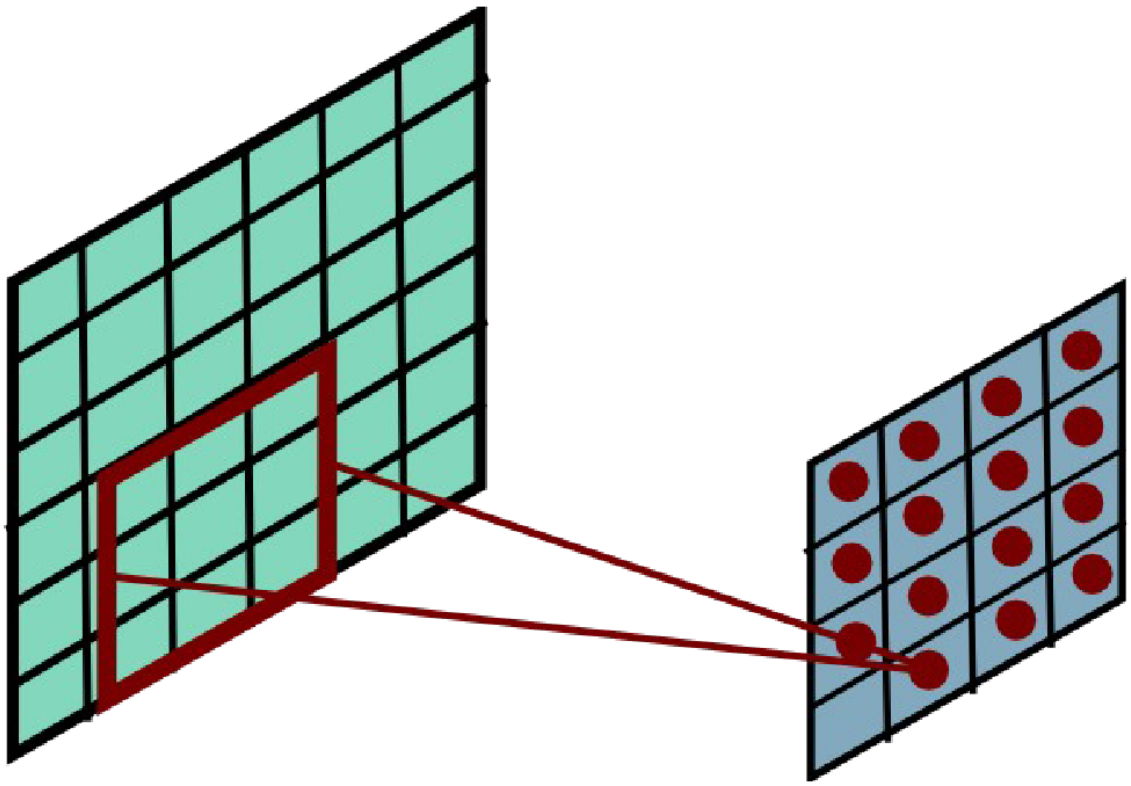


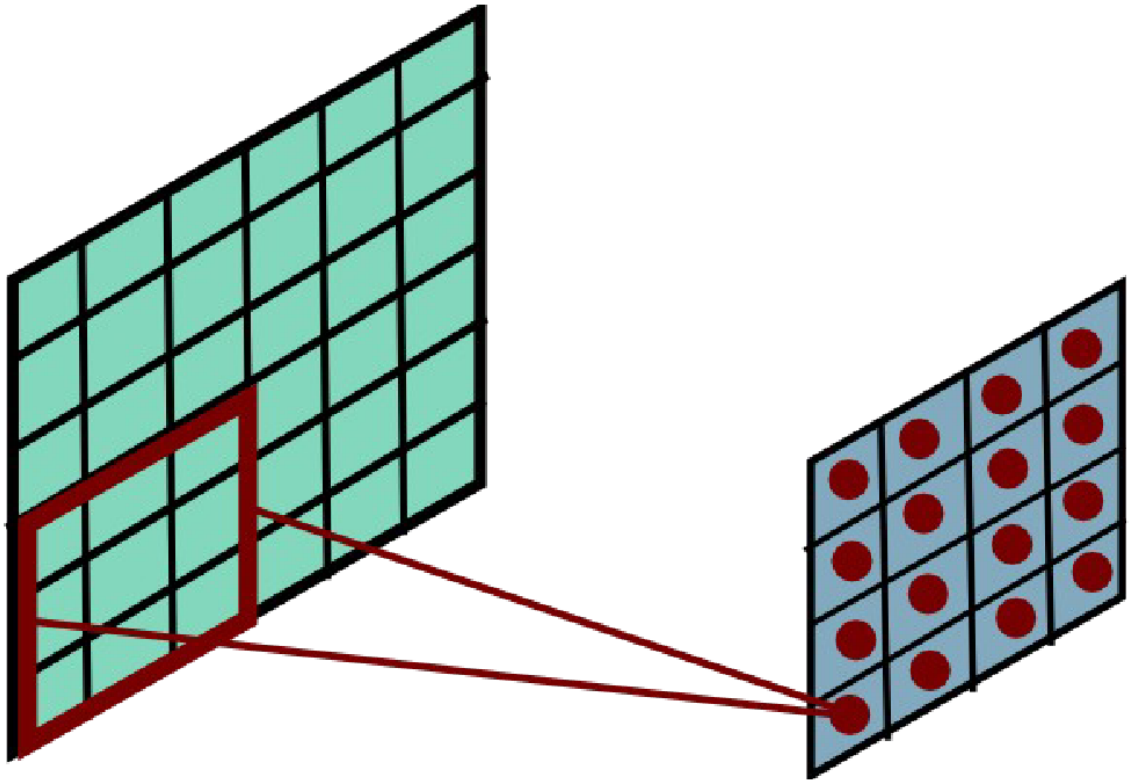




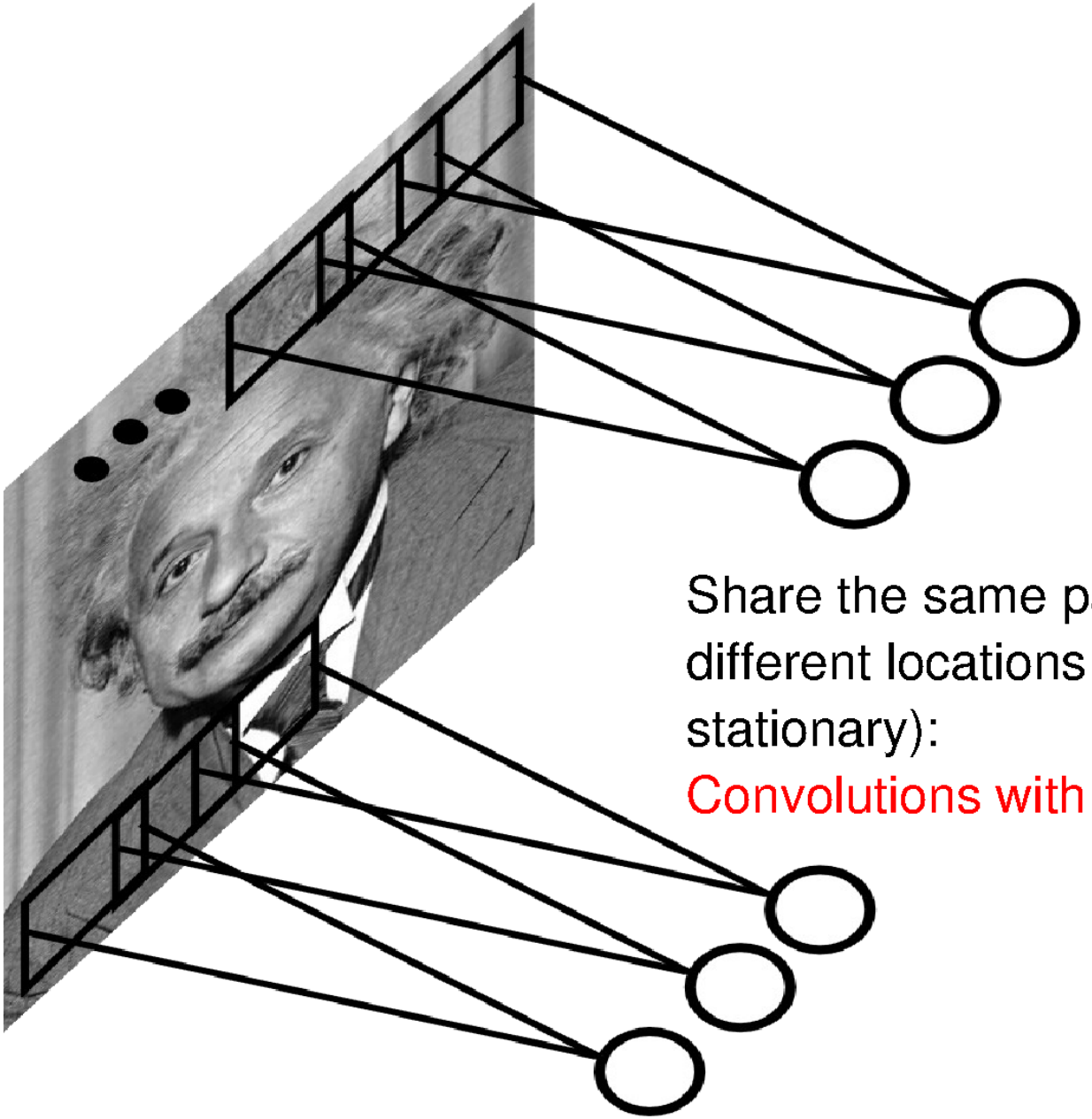








Why Convolution?

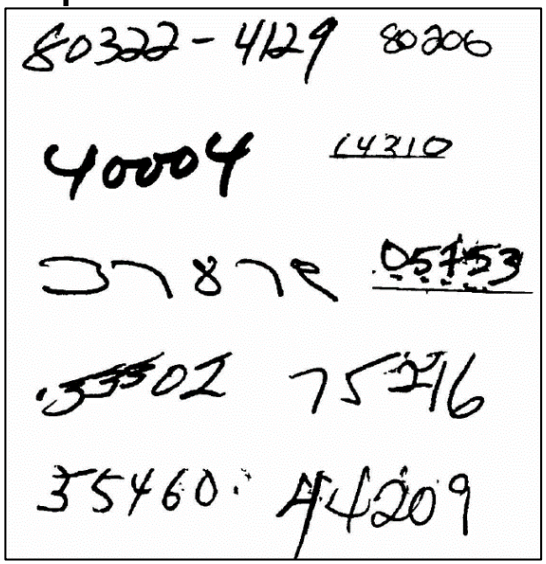


Share the same parameters across different locations (assuming input is stationary):

Convolutions with learned kernels

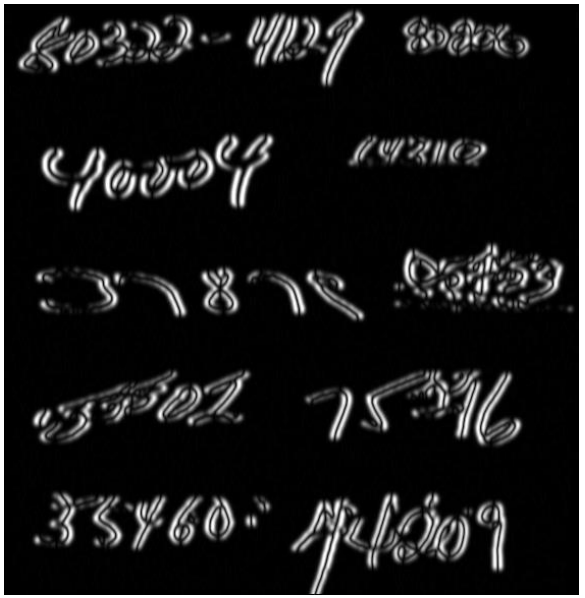
Why Convolution?

Input



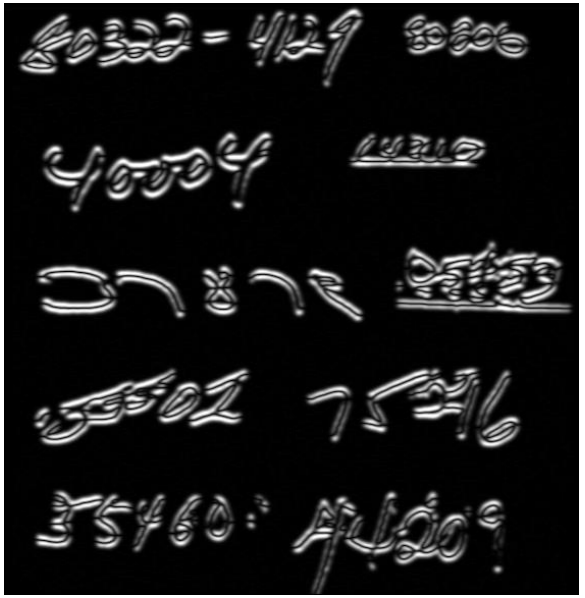
-1	0	+1
-2	0	+2
-1	0	+1

x filter



+1	+2	+1
0	0	0
-1	-2	-1

y filter



Local Template Matching

Convolution: Formal Definition

- Given array u_t and w_t, their convolution is a function s_t

$$s_{x,y} = \sum_{a=-\infty}^{+\infty} \sum_{b=-\infty}^{+\infty} u_{a,b}\, w_{x-a,\, y-b}$$

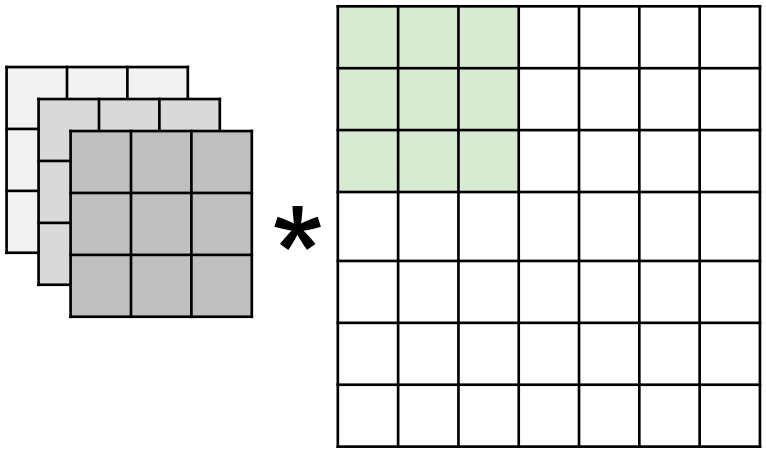
$$s = w * u = u * w$$

- When u_t or w_t is not defined, assumed to be 0

- w_t is called the convolution filter, or convolution kernel

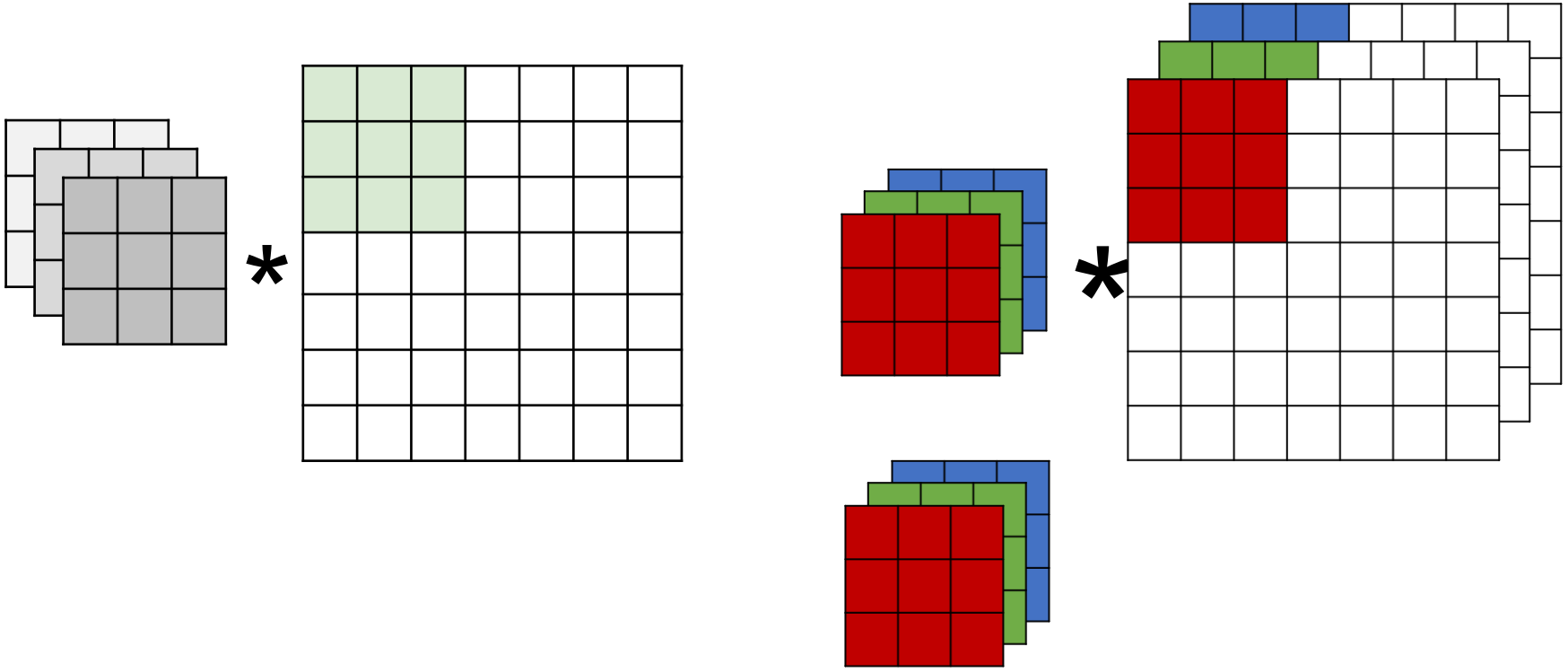
Convolution with multiple filters

- Multiple kernels can be used

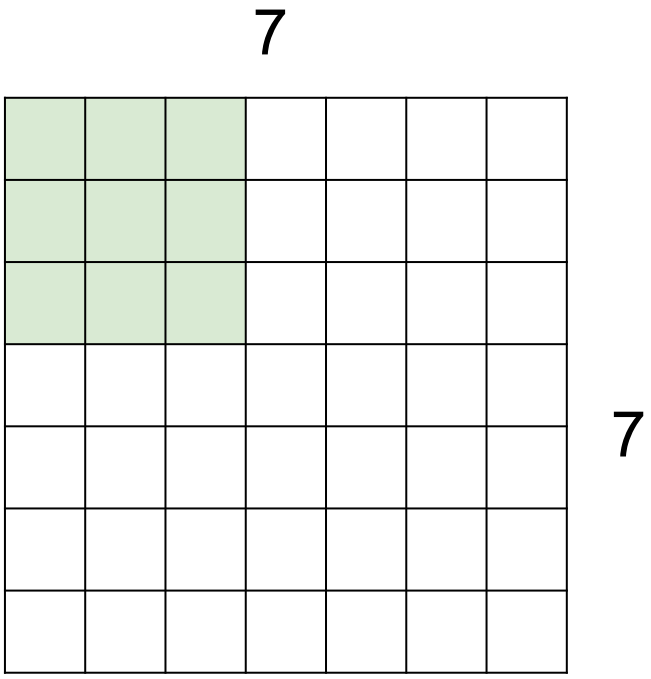


Convolution with multiple filters

- Multiple kernels can be used
- Input / kernel can be multidimensional (2D / 3D ...)

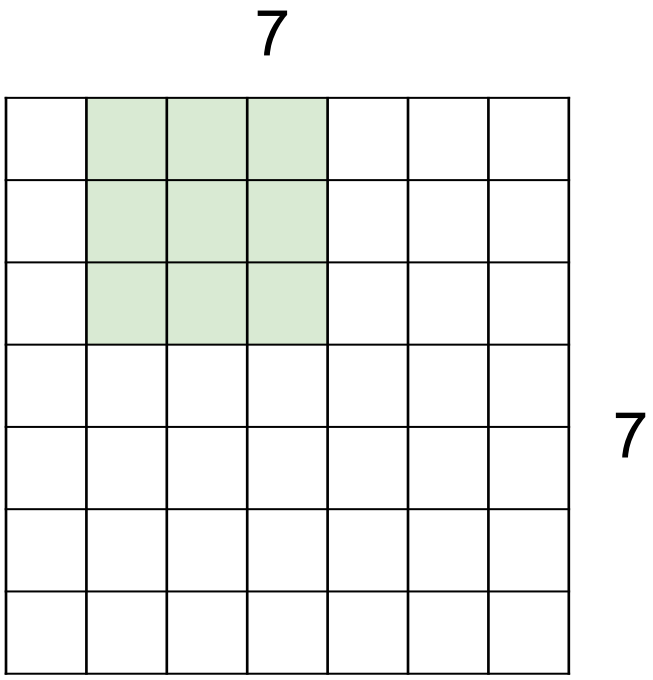


Convolution: Spatial Dimension



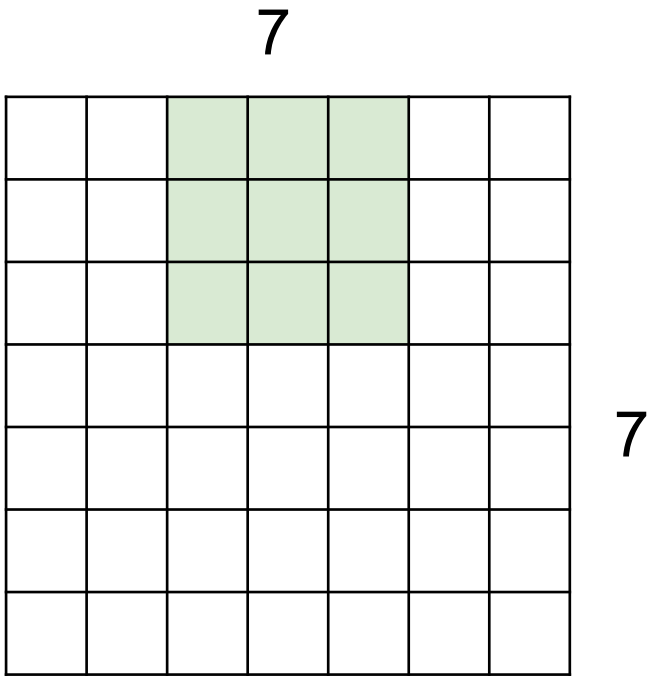
7x7 input (spatially)
assume 3x3 filter

Convolution: Spatial Dimension



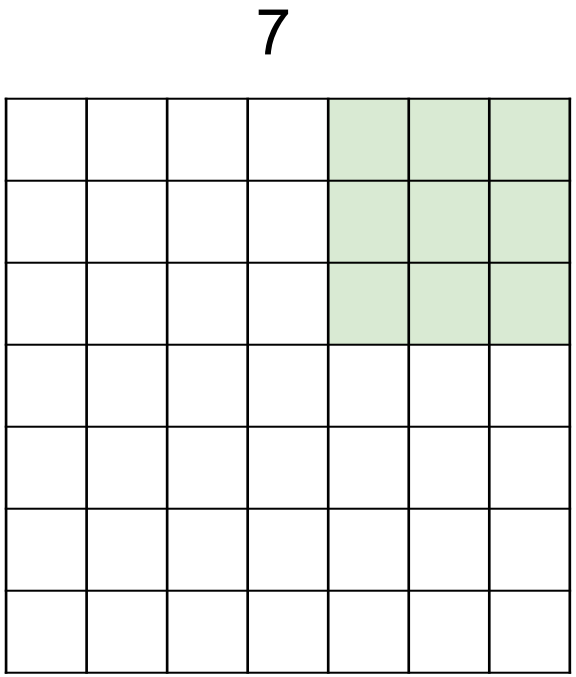
7x7 input (spatially)
assume 3x3 filter

Convolution: Spatial Dimension



7x7 input (spatially)
assume 3x3 filter

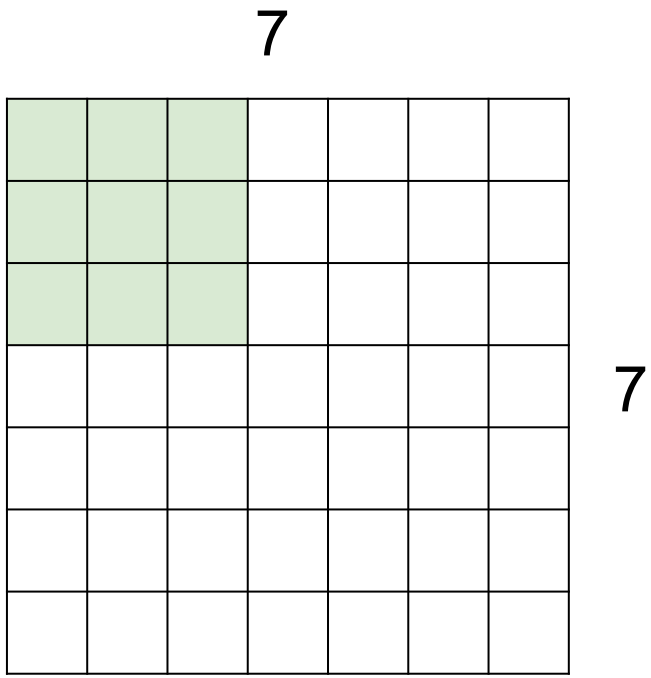
Convolution: Spatial Dimension



7x7 input (spatially)
assume 3x3 filter

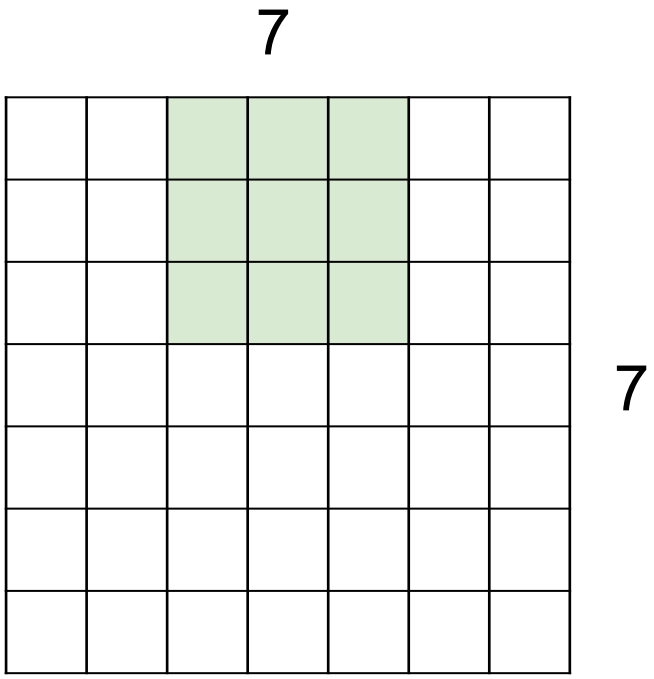
=> 5x5 output

Convolution: Spatial Dimension



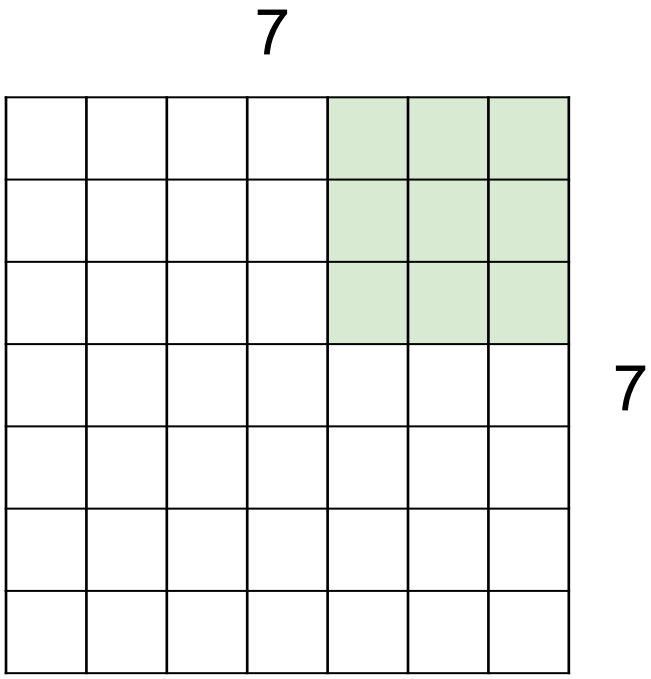
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

Convolution: Spatial Dimension



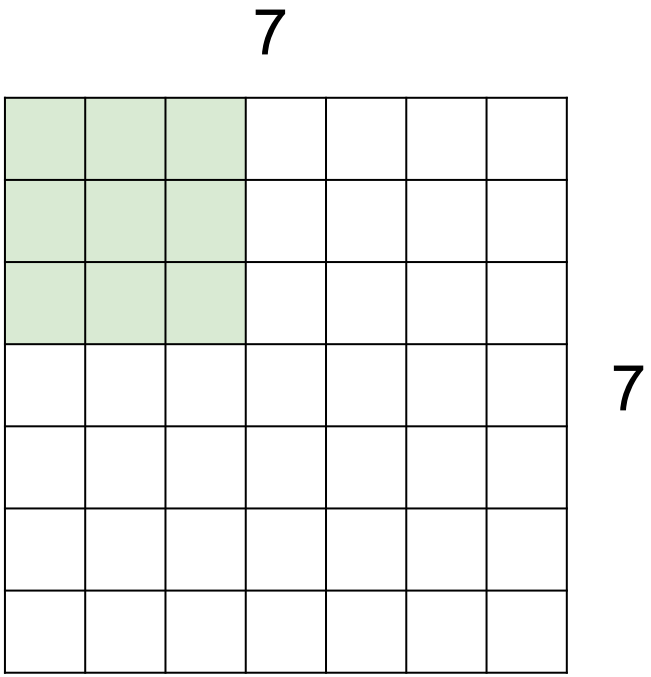
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

Convolution: Spatial Dimension



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output

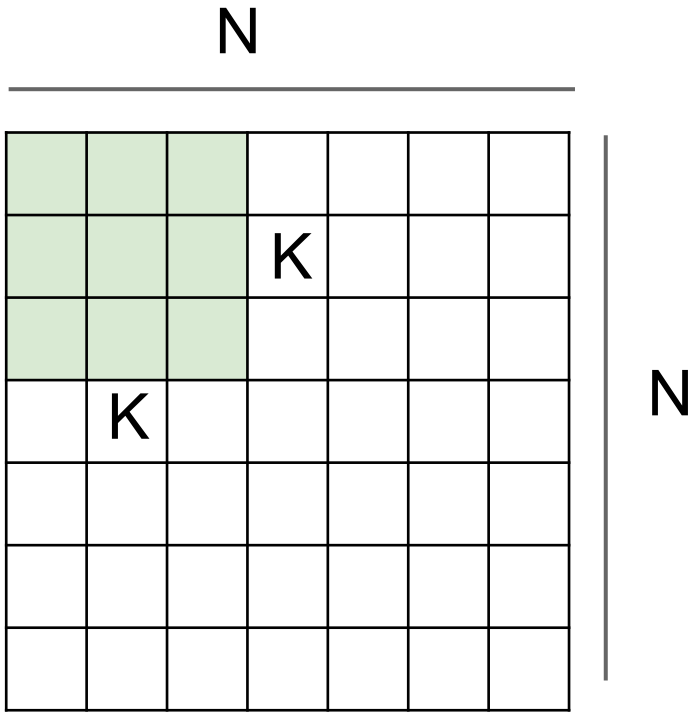
Convolution: Spatial Dimension



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3**

doesn't quite fit!

Convolution: Spatial Dimension

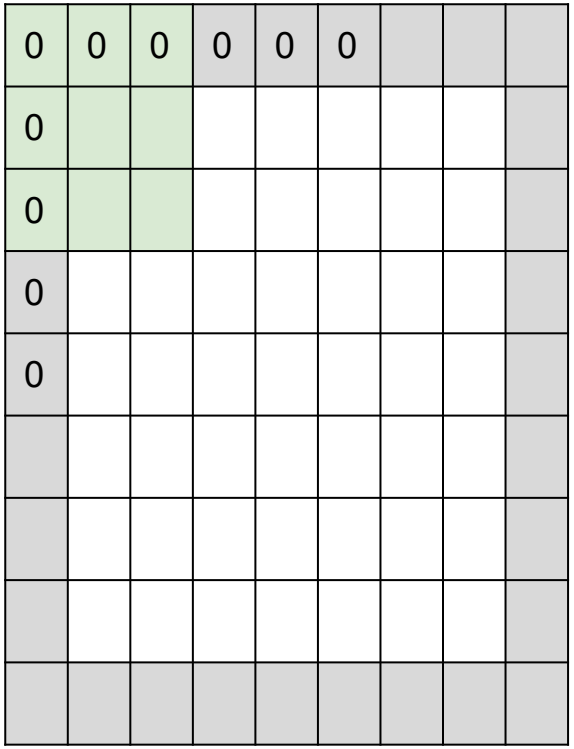


Output size:
(N - K) / stride + 1

e.g. N = 7, K = 3:
stride 1 => (7 - 3)/1 + 1 = 5
stride 2 => (7 - 3)/2 + 1 = 3
stride 3 => (7 - 3)/3 + 1 = 2 ☹

Convolution: Spatial Dimension

In practice: Common to zero pad the border



e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

(N – K + 2P) / stride + 1

Convolution: Forward propagation and backpropagation

Question: How to compute the gradients in theory?

Hint: Convolutions are linear operations

Let us consider a 1D example $y = h * x$

$$y = h * x = \begin{bmatrix} h_1 & 0 & \dots & 0 & 0 \\ h_2 & h_1 & \dots & \vdots & \vdots \\ h_3 & h_2 & \dots & 0 & 0 \\ \vdots & h_3 & \dots & h_1 & 0 \\ h_{m-1} & \vdots & \dots & h_2 & h_1 \\ h_m & h_{m-1} & \vdots & \vdots & h_2 \\ 0 & h_m & \dots & h_{m-2} & \vdots \\ 0 & 0 & \dots & h_{m-1} & h_{m-2} \\ \vdots & \vdots & \vdots & h_m & h_{m-1} \\ 0 & 0 & 0 & \dots & h_m \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix}$$

H Toeplitz matrix

$$y = Hx$$

$$\frac{dy}{dx} = H$$

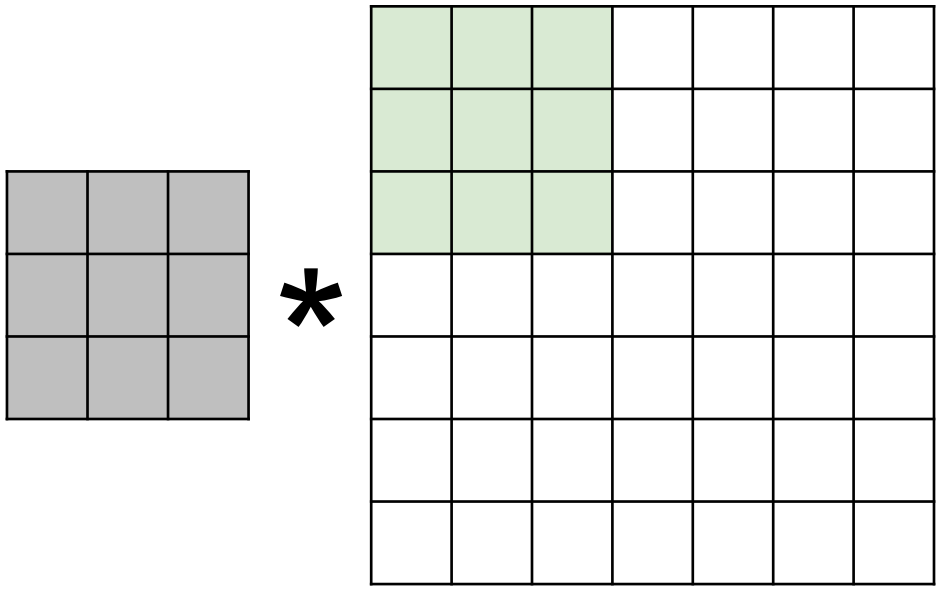
$$\frac{dy}{d\boldsymbol{vec}(H)} = I \otimes x^T$$

* This is the actual **convolution**

Convolution:
Forward propagation and backpropagation

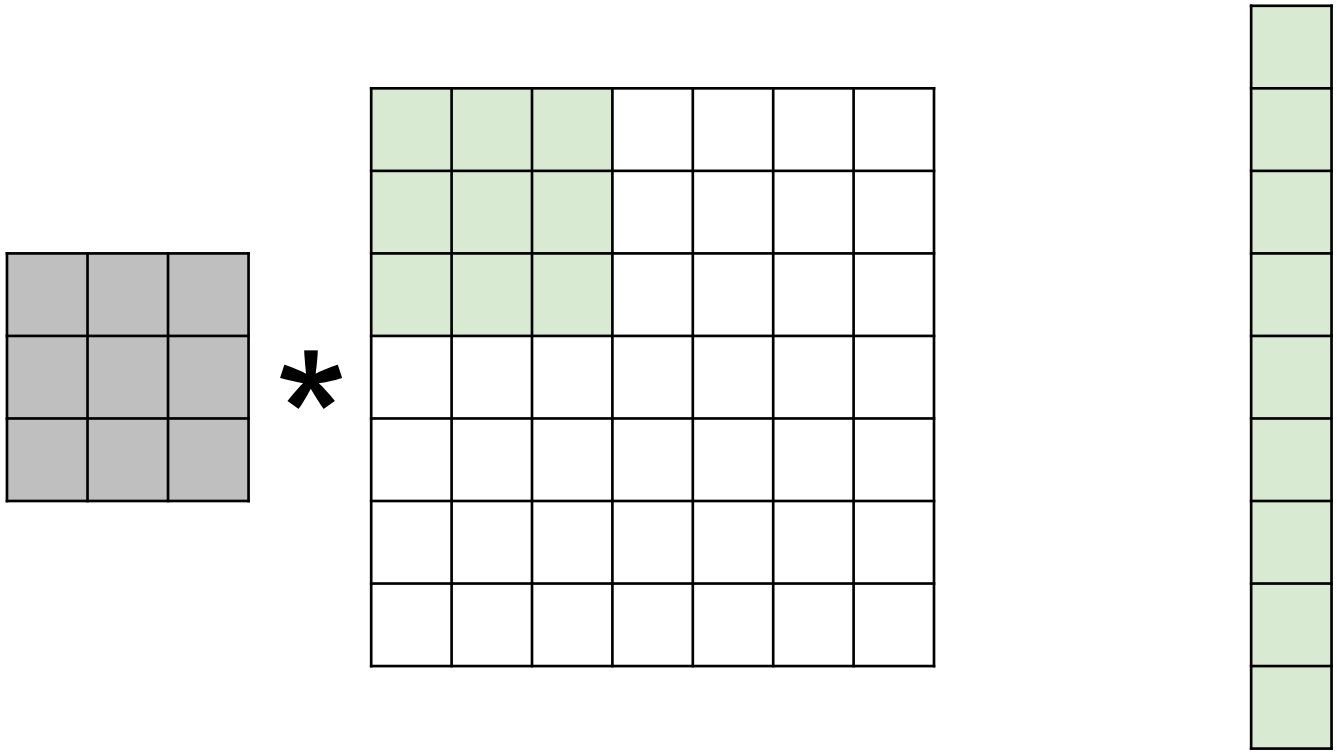
Question: How to compute the gradients in practice?

Hint: Unfold the image & use efficient matrix multiplication



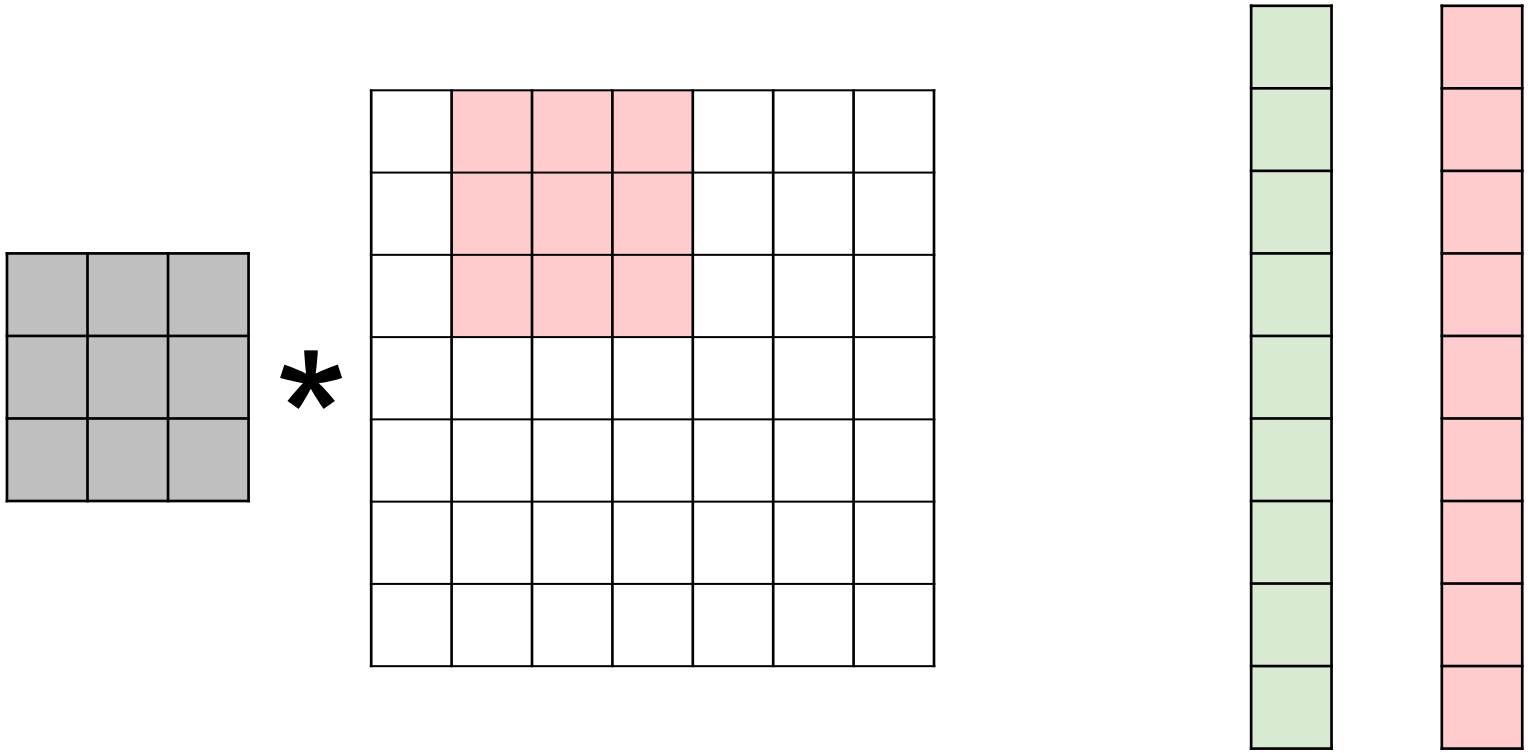
Convolution: Forward Propagation

Unfold the image & use efficient matrix multiplication



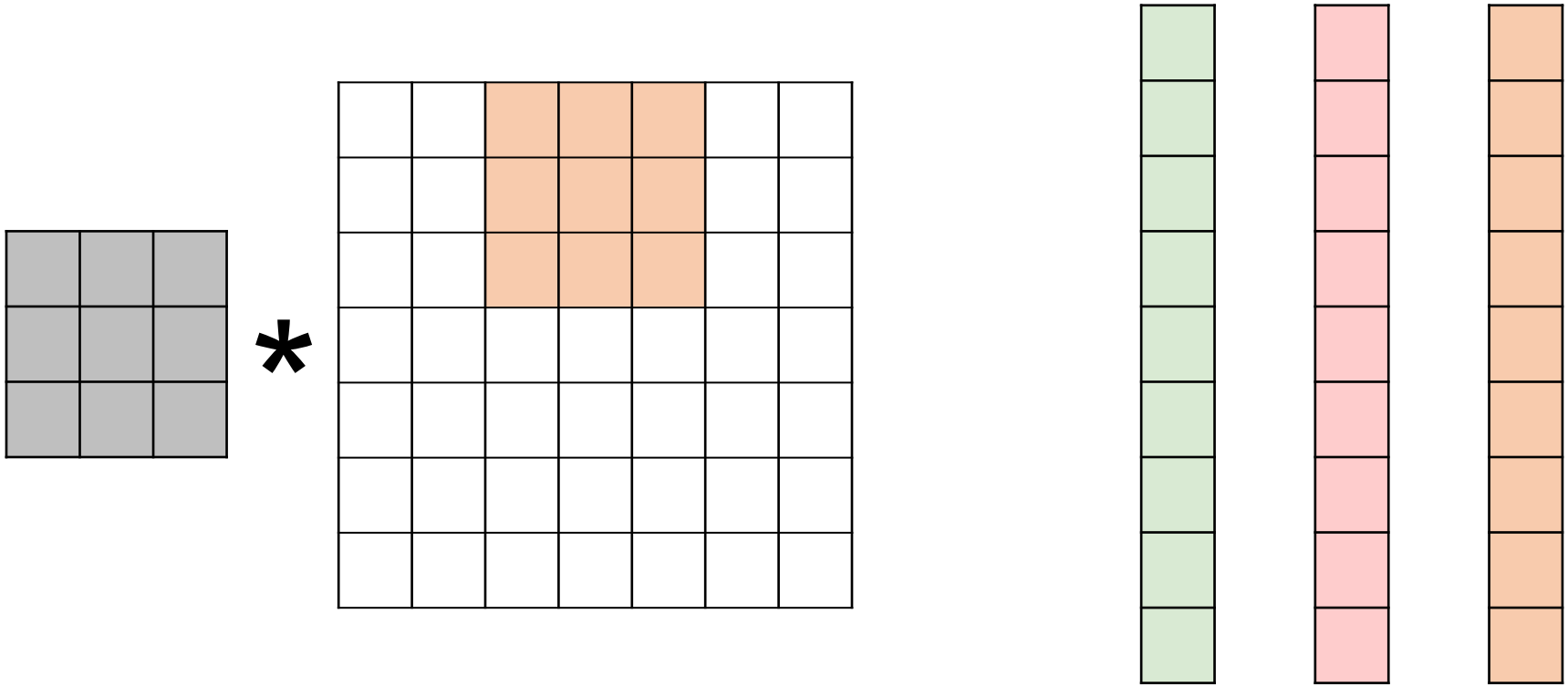
Convolution: Forward Propagation

Unfold the image & use efficient matrix multiplication



Convolution: Forward Propagation

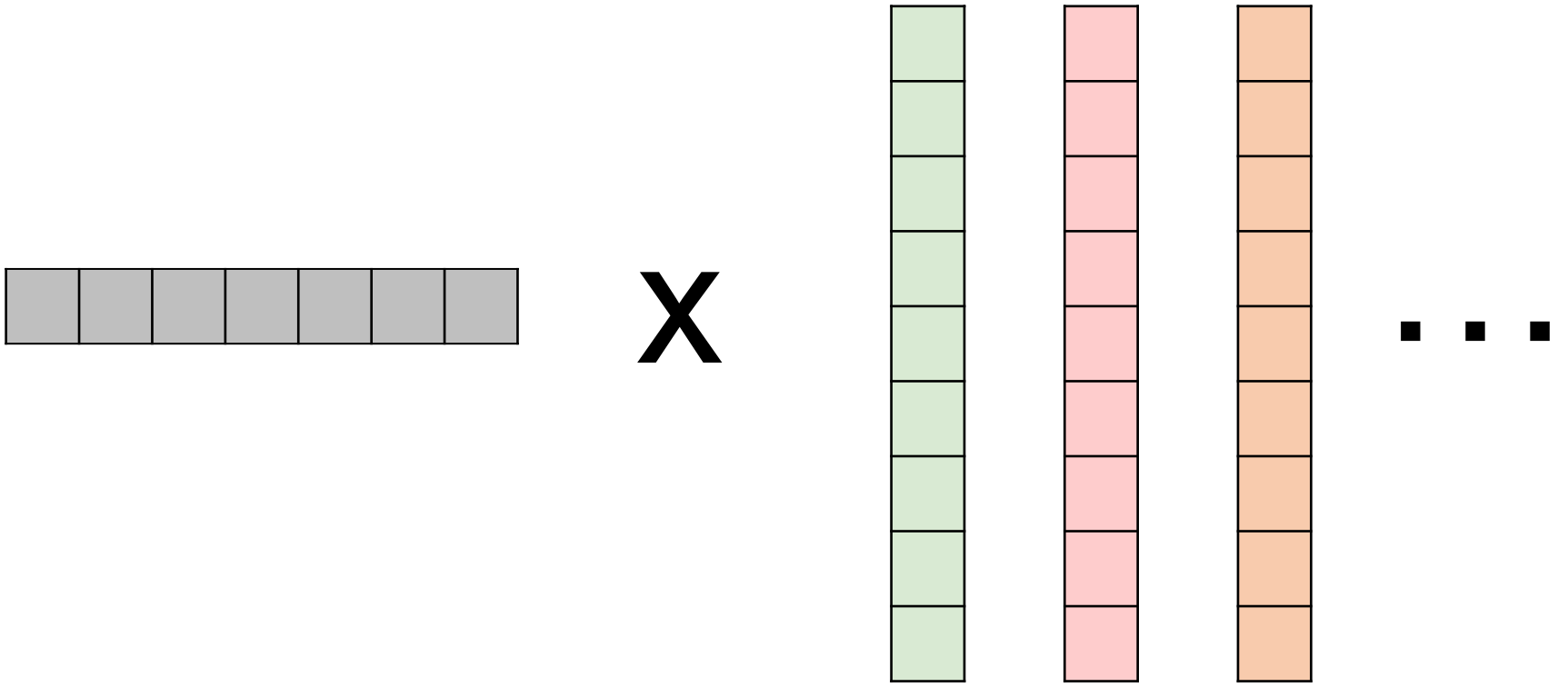
Unfold the image & use efficient matrix multiplication



Convolution: Forward Propagation

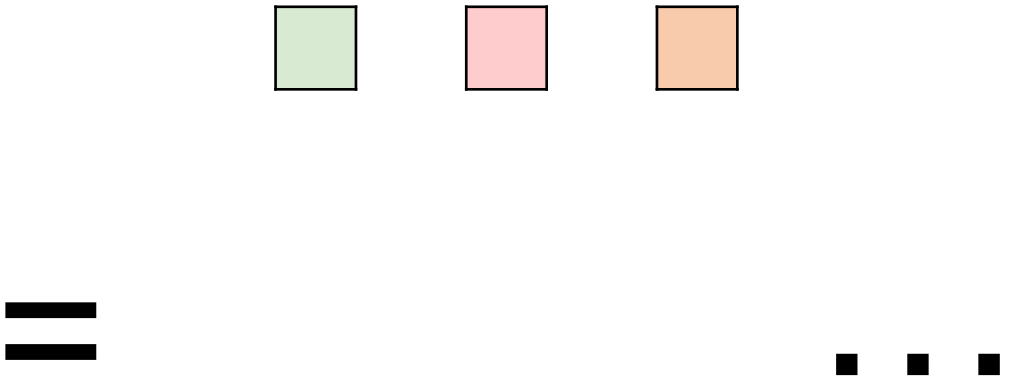
Unfold the image & use efficient matrix multiplication

Unfold: Can have arbitrary strides



Convolution: Forward Propagation

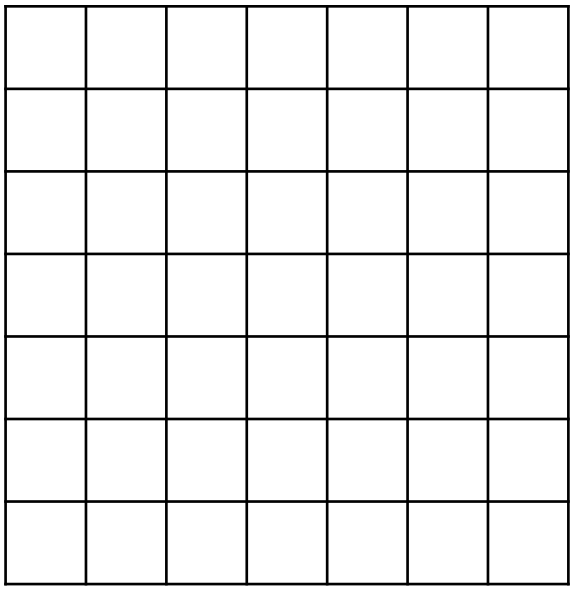
Unfold the image & use efficient matrix multiplication

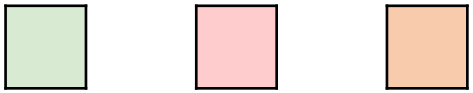


Convolution: Forward Propagation

Fold the vectors back into an image

Fold: Can have arbitrary strides



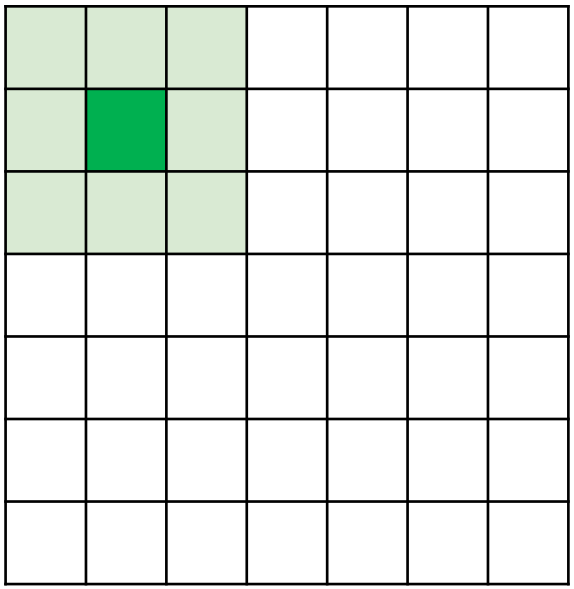


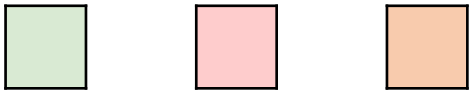


Convolution: Forward Propagation

Fold the vectors back into an image

Fold: Can have arbitrary strides



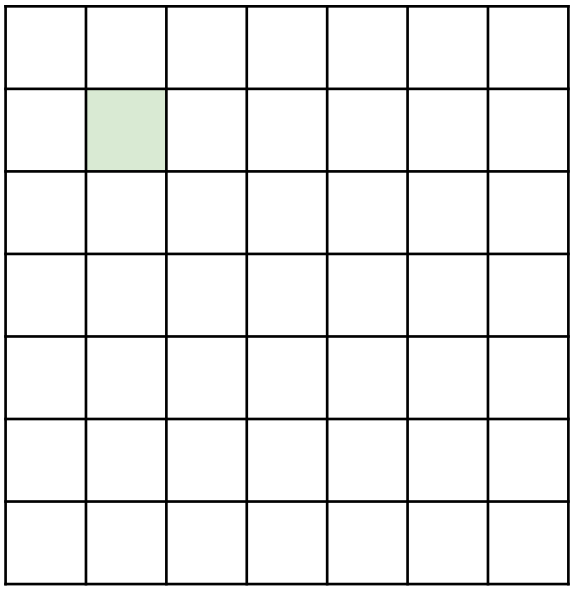


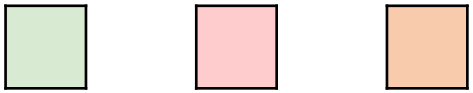


Convolution: Forward Propagation

Fold the vectors back into an image

Fold: Can have arbitrary strides



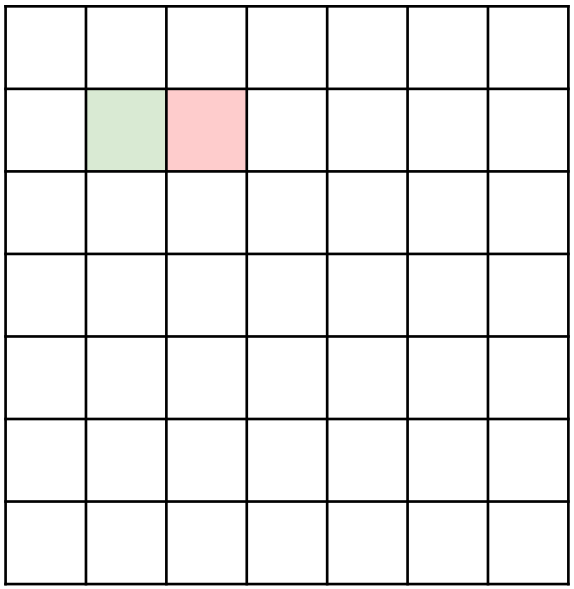


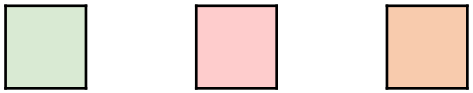


Convolution: Forward Propagation

Fold the vectors back into an image

Fold: Can have arbitrary strides



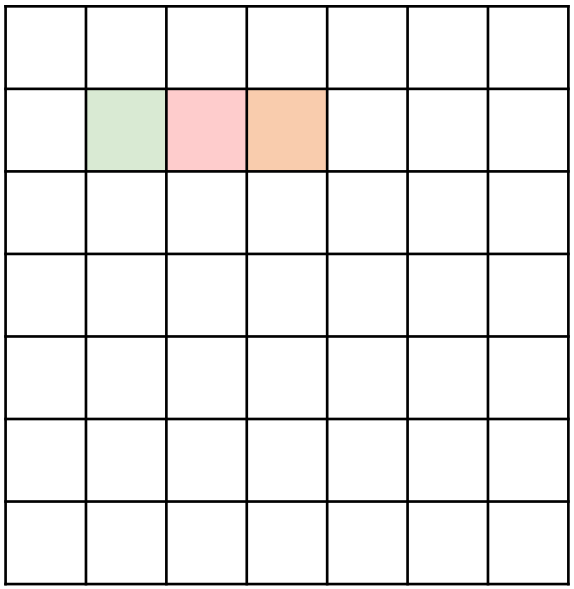


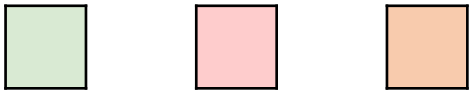


Convolution: Forward Propagation

Fold the vectors back into an image

Fold: Can have arbitrary strides

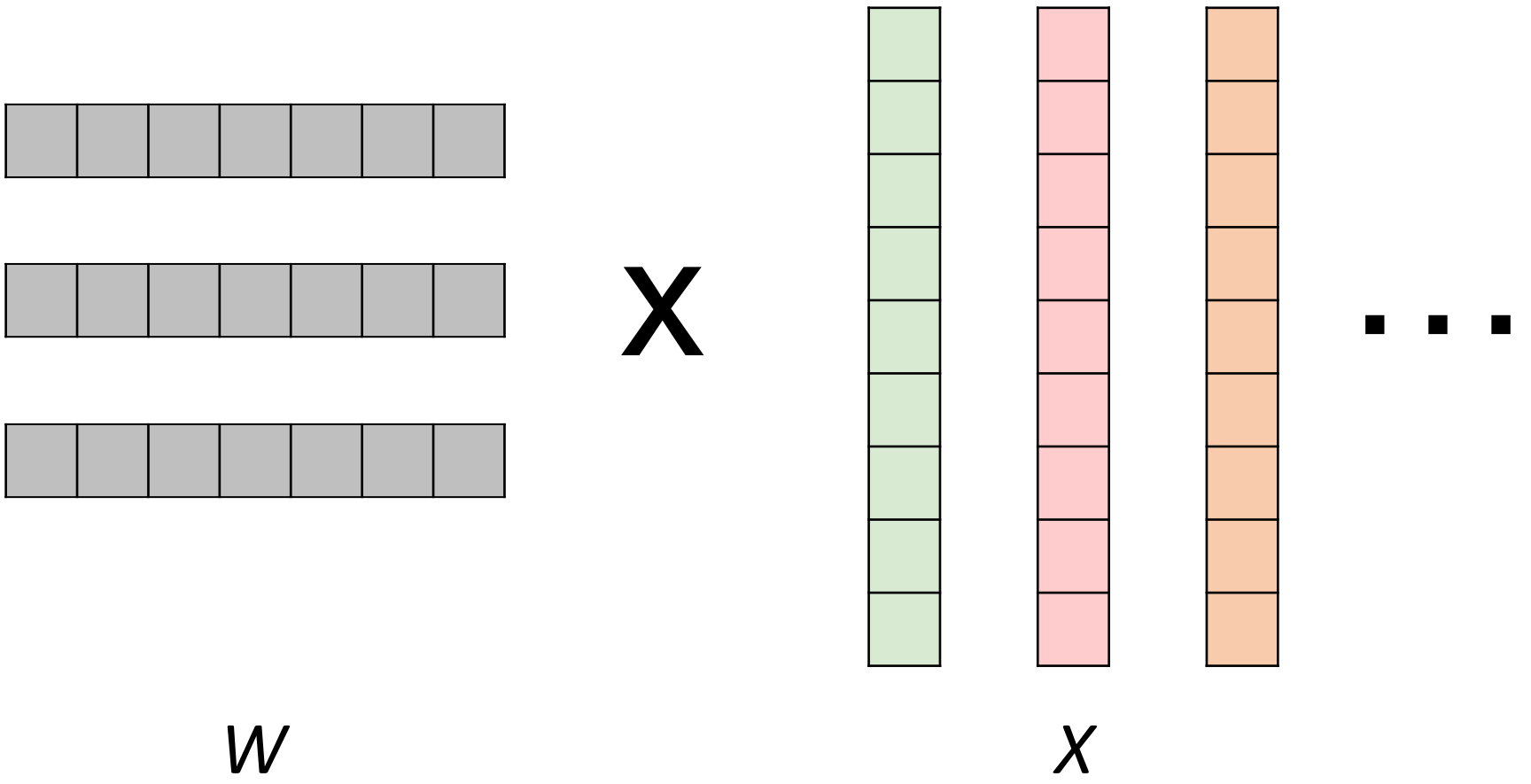






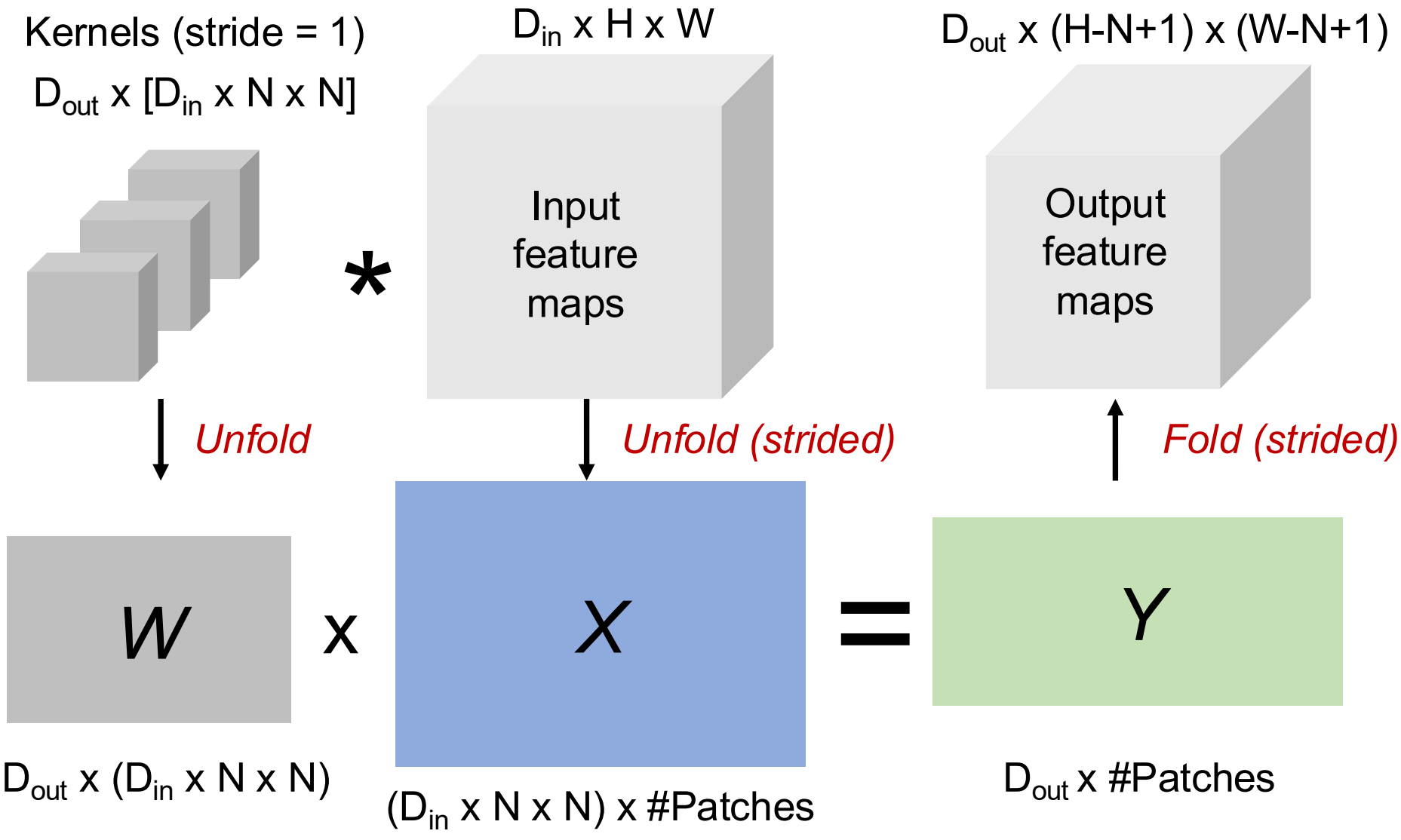
Convolution: Forward Propagation

Unfold the image & use efficient matrix multiplication



Convolution: Forward Propagation

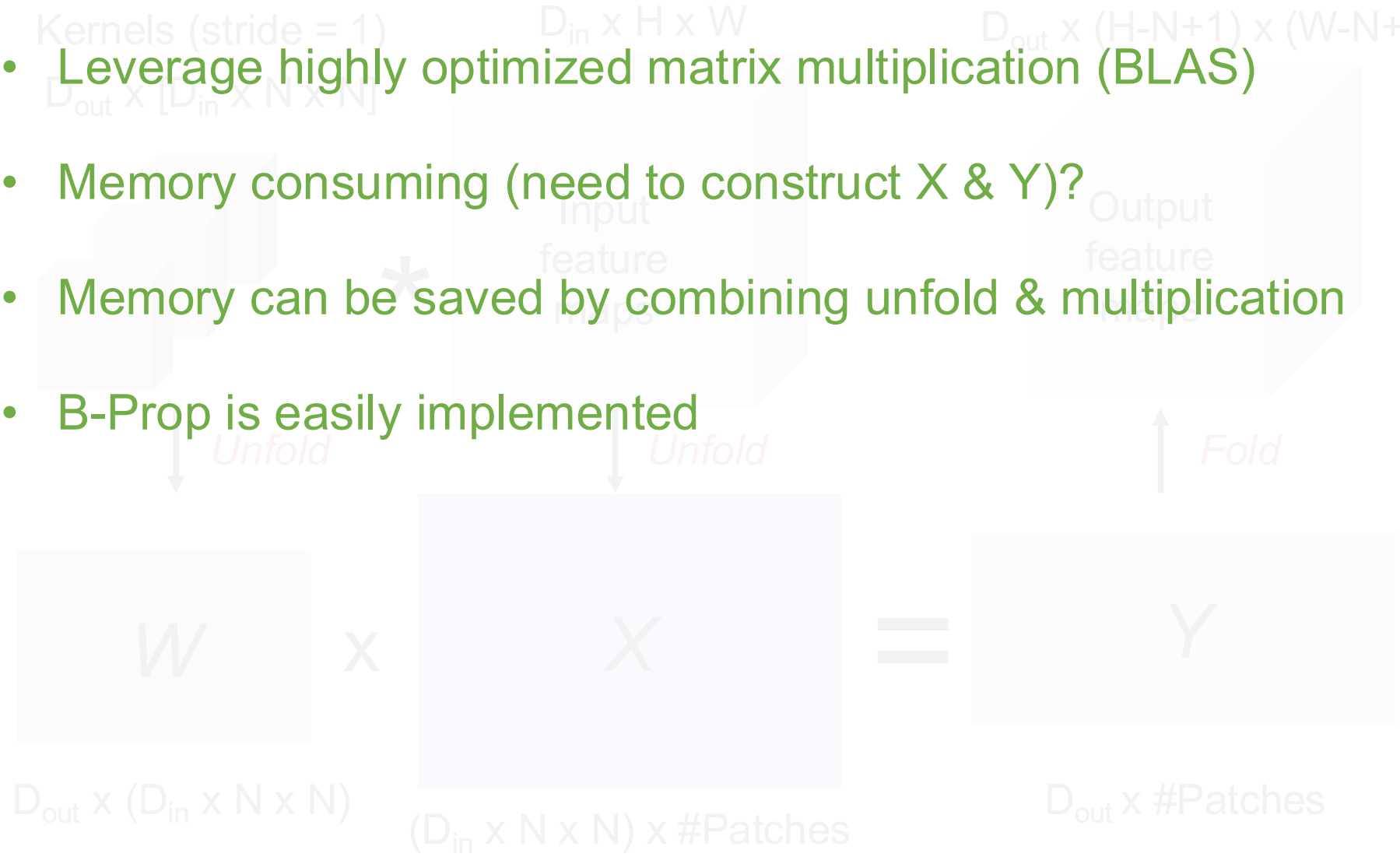
F-Prop: Unfold the image & use efficient matrix multiplication



Convolution: Forward Propagation

F-Prop: Unfold the image & use efficient matrix multiplication

- Leverage highly optimized matrix multiplication (BLAS)
- Memory consuming (need to construct X & Y)?
- Memory can be saved by combining unfold & multiplication
- B-Prop is easily implemented

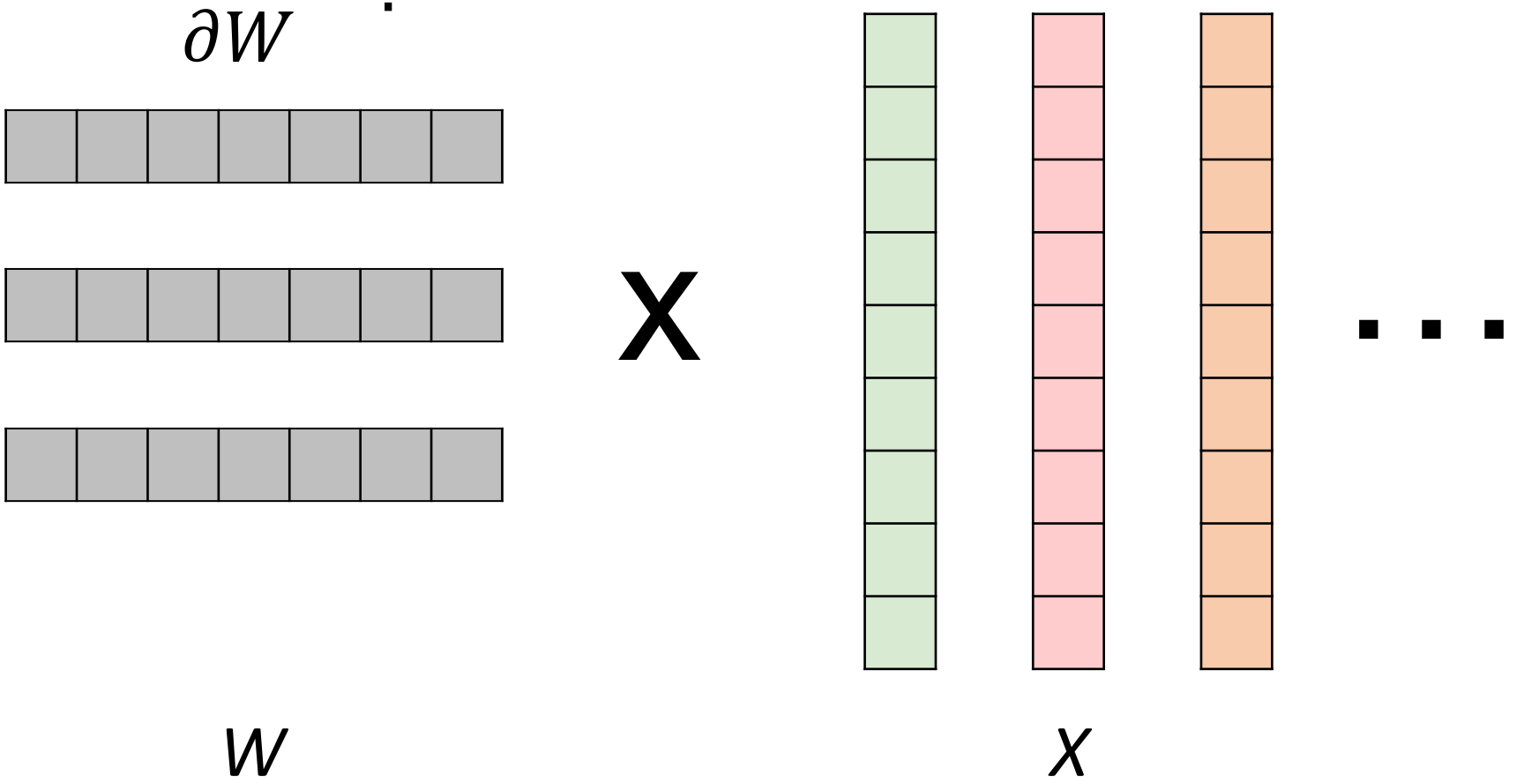


Convolution: Backward Propagation

Unfold the image & use efficient matrix multiplication

$$\frac{\partial Y}{\partial W} = ?$$

Unfold: Can have arbitrary strides



W X

Convolution: Backward Propagation

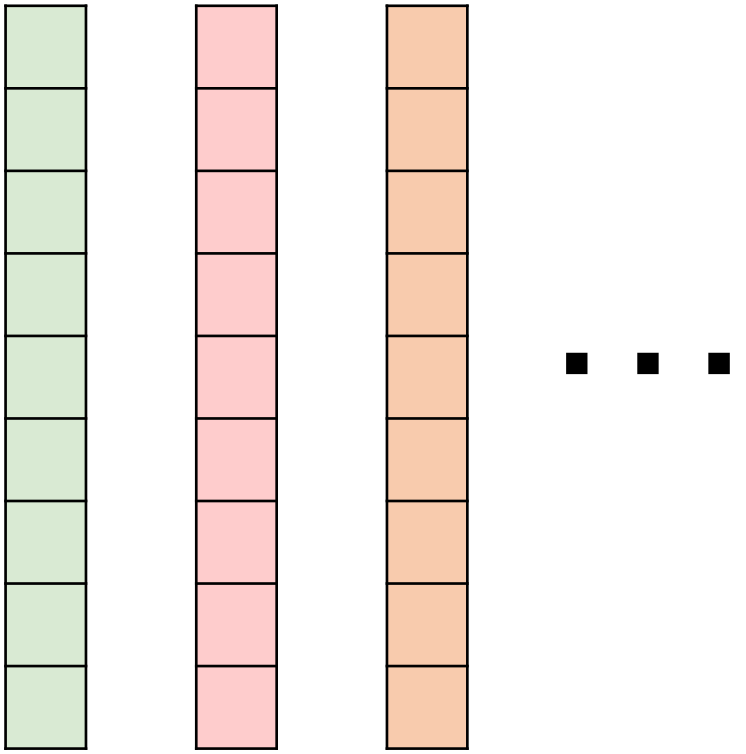
Unfold the image & use efficient matrix multiplication

$$\frac{\partial Y}{\partial \boldsymbol{w}^T} = ?$$



X

Unfold: Can have arbitrary strides

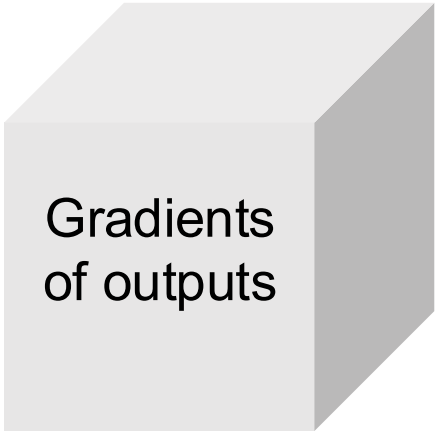


X

Convolution: Backpropagation

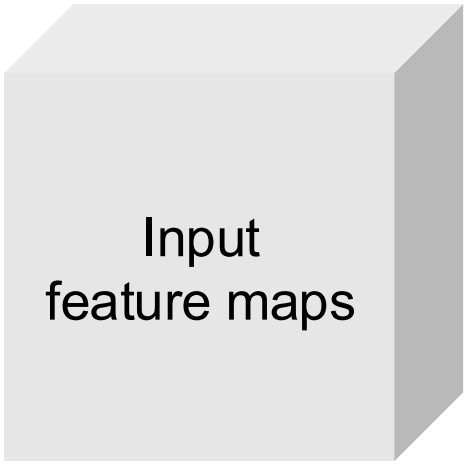
B-Prop: Compute gradients w.r.t. parameters

D_{out} x (H-N+1) x (W-N+1)



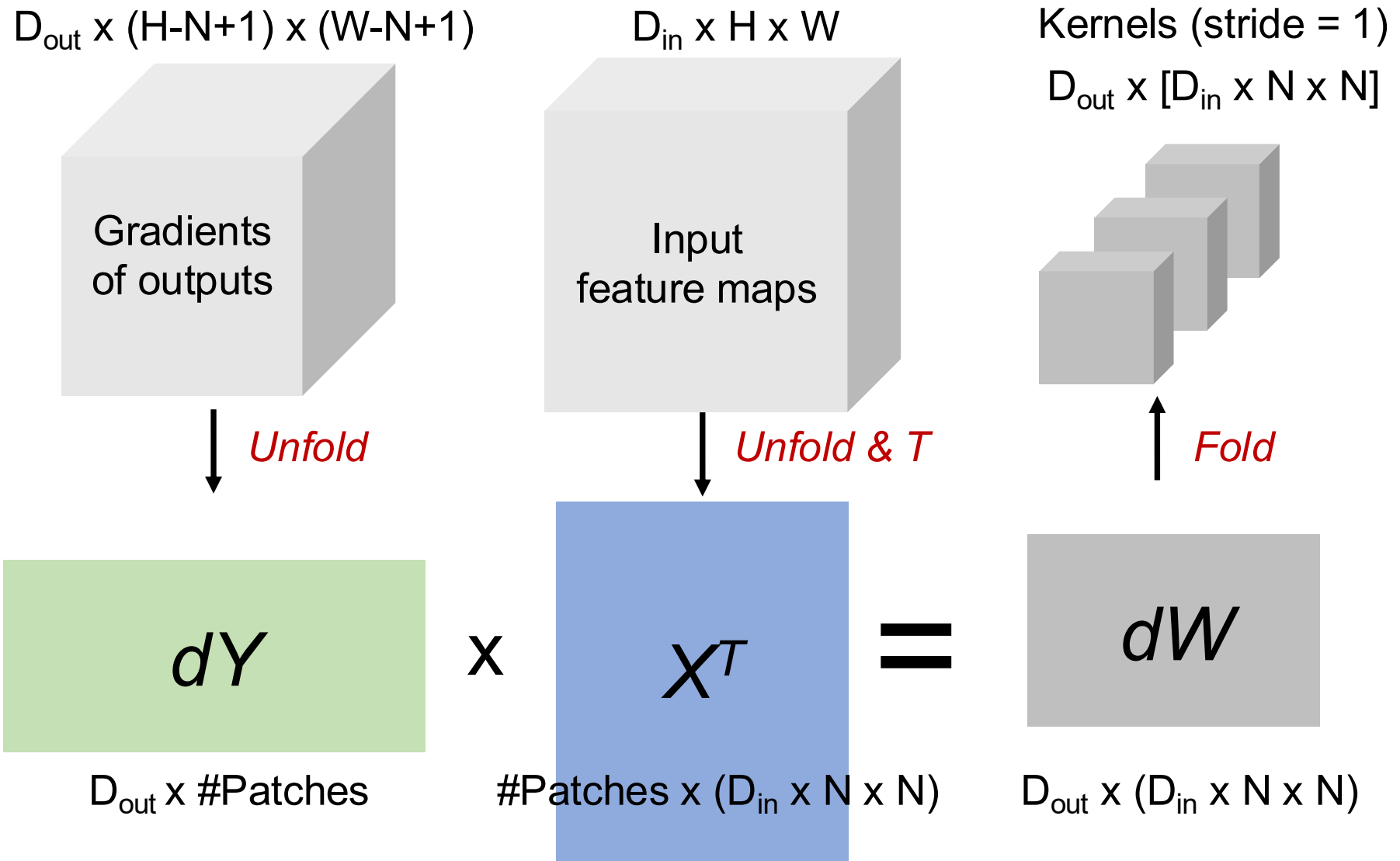
Assuming a scalar loss

D_{in} x H x W



Convolution: Backpropagation

B-Prop: Compute gradients w.r.t. parameters

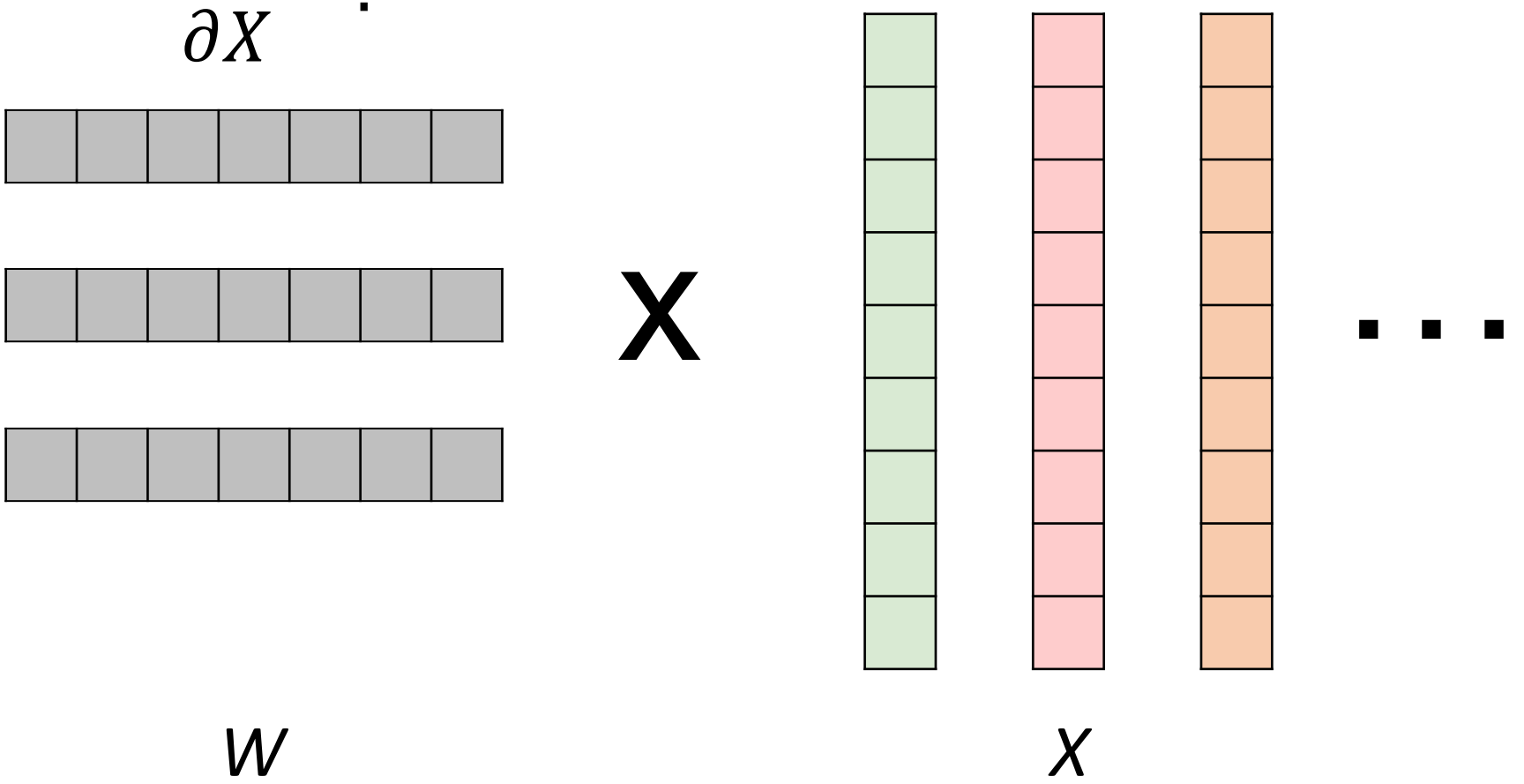


Convolution: Backward Propagation

Unfold the image & use efficient matrix multiplication

$$\frac{\partial Y}{\partial X} = ?$$

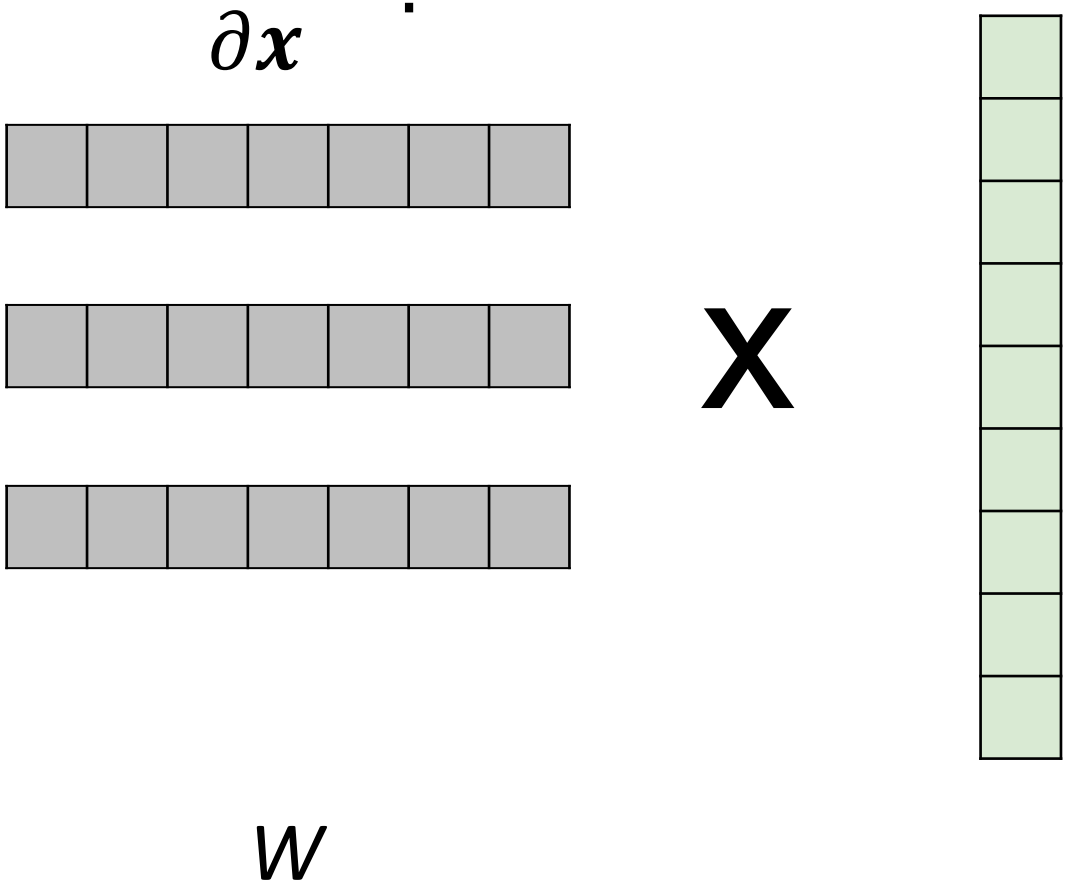
Unfold: Can have arbitrary strides



Convolution: Backward Propagation

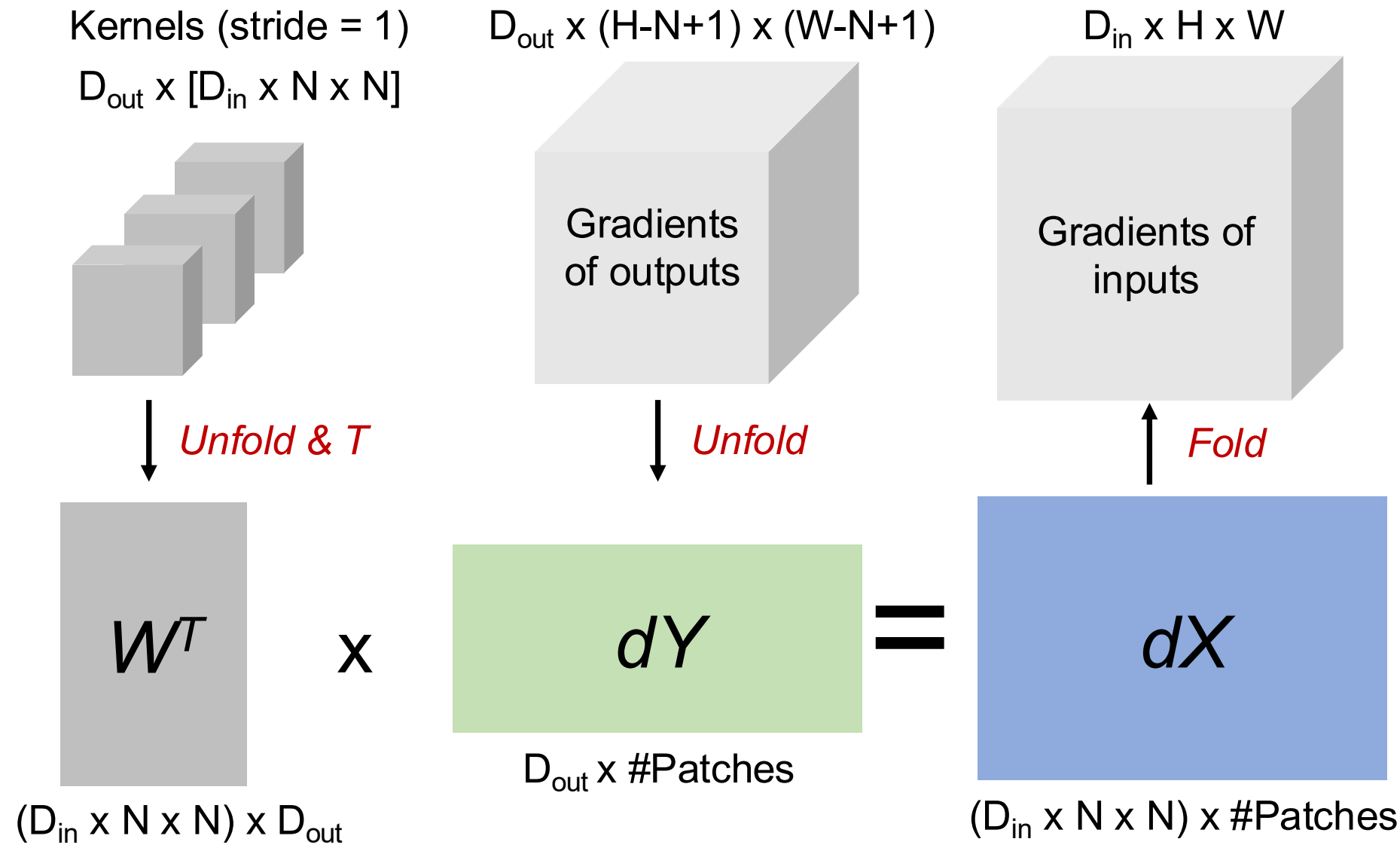
Unfold the image & use efficient matrix multiplication

$$\frac{\partial Y}{\partial \boldsymbol{x}} = ?$$



Convolution: Backpropagation

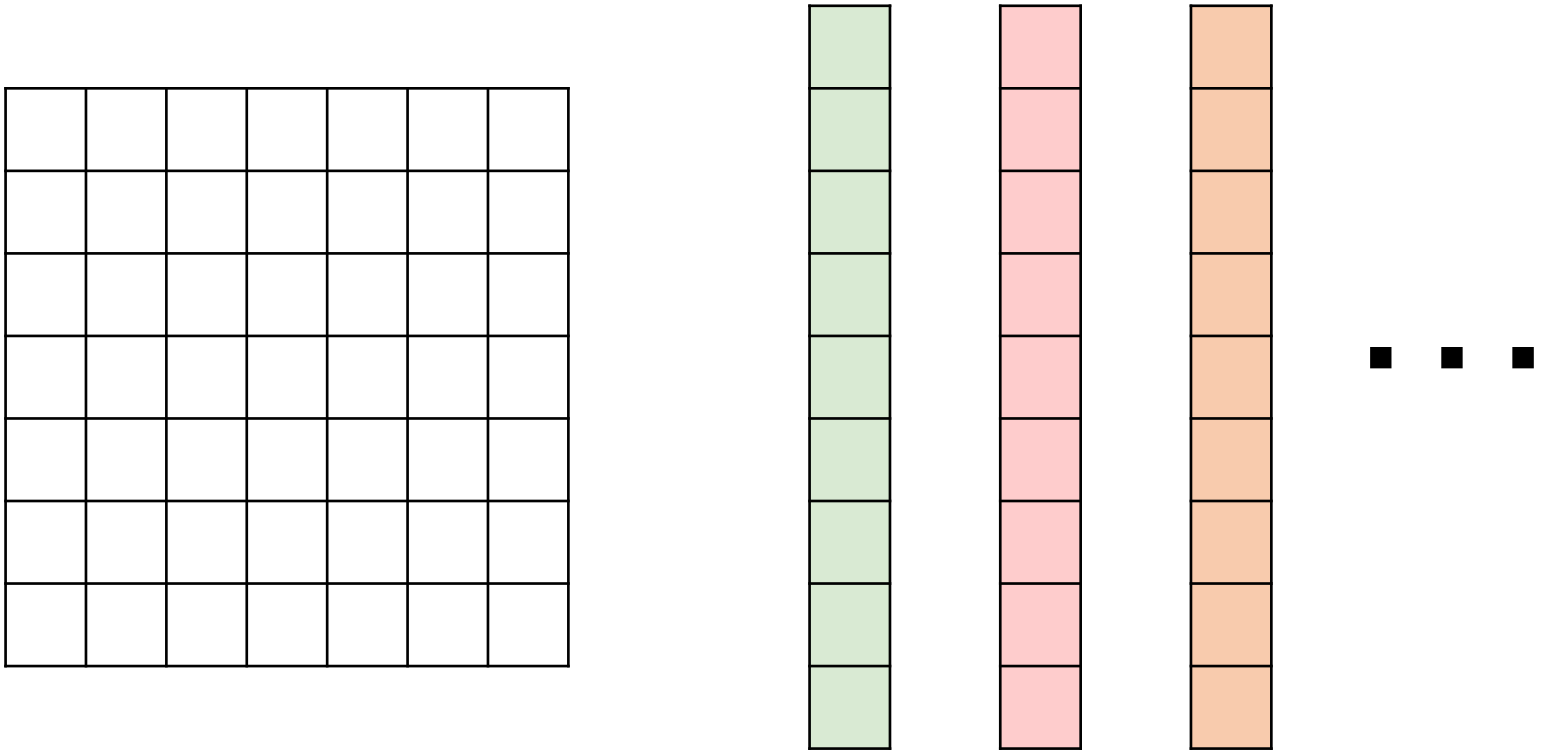
B-Prop: Compute gradients w.r.t. inputs



Convolution: Backpropagation

Fold the vectors back into an image and sum them up

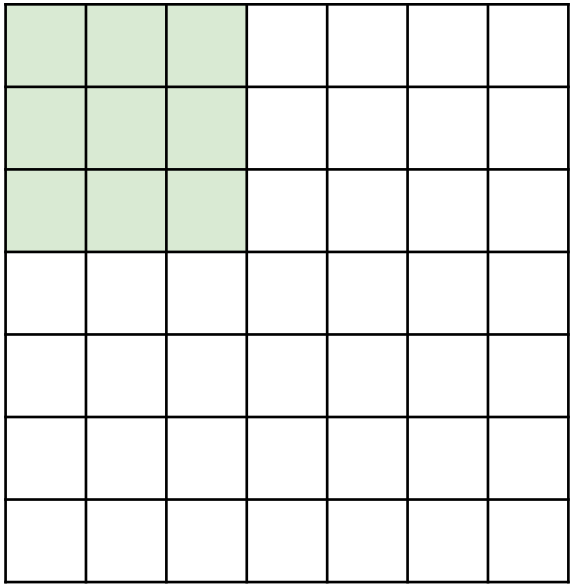
Fold: Can have arbitrary strides

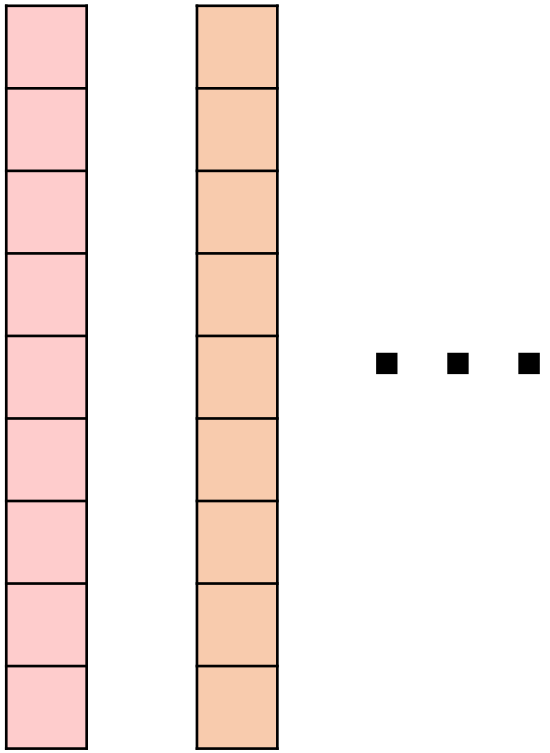


Convolution: Backpropagation

Fold the vectors back into an image and sum them up

Fold: Can have arbitrary strides

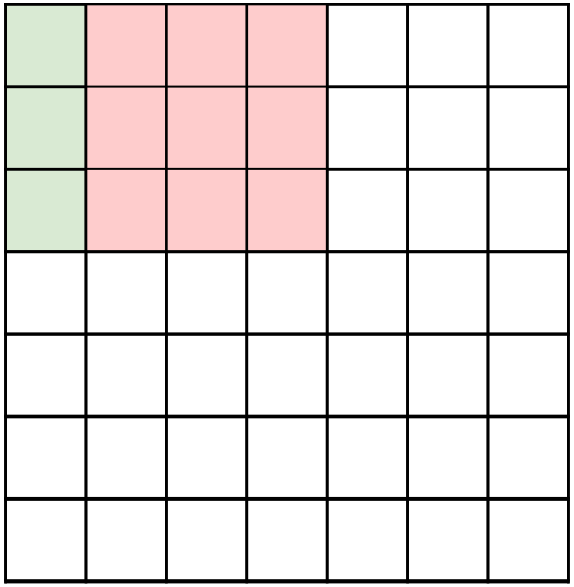


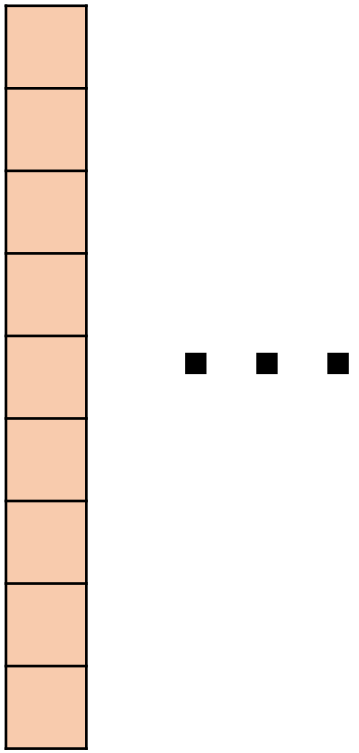


Convolution: Backpropagation

Fold the vectors back into an image and sum them up

Fold: Can have arbitrary strides

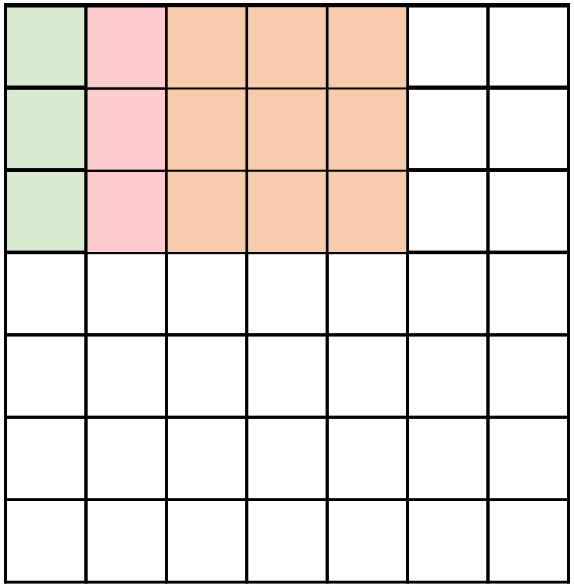




Convolution: Backpropagation

Fold the vectors back into an image and sum them up

Fold: Can have arbitrary strides



. . .

Other types of convolutions

Transposed Convolution (Deconvolution?)

- Up-sampling of the responses
- Often used for semantic segmentation (more later …)

Grouped Convolution

- Multi-branch convolutions
- Reduce complexity

Depthwise Convolution

- Channel-wise convolution (special type of grouped conv)
- Reduce complexity

Dilated Convolution

- Convolution kernels with holes (preserve resolution)
- Used for semantic segmentation

Convolution: Summary

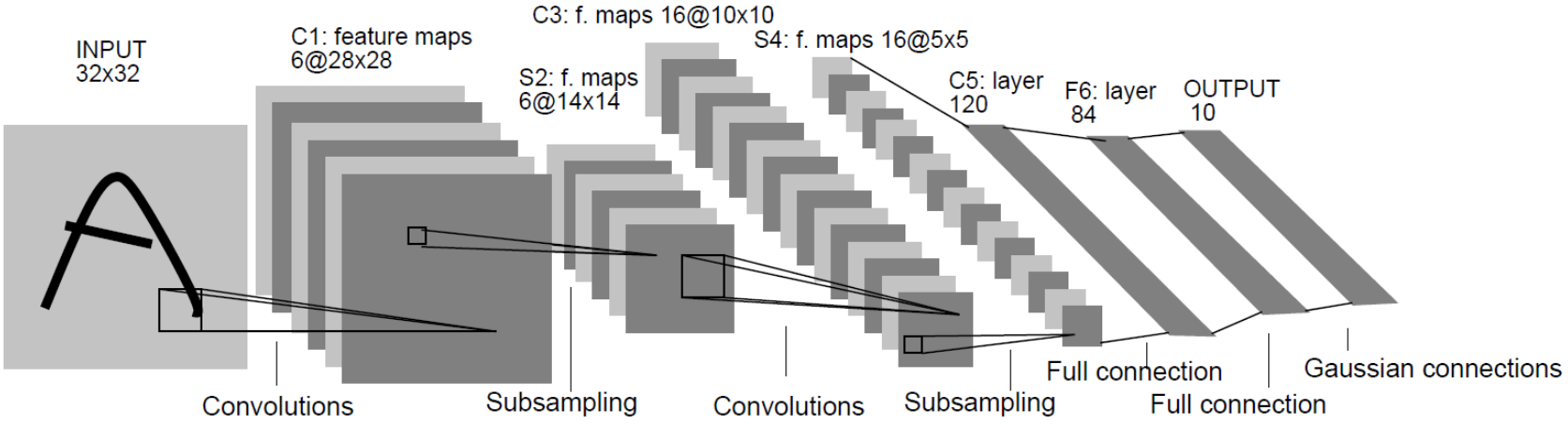
A convolution operation

- connects each hidden unit to a small patch of the input
- shares the weight across space
- remains a linear function!

A neural network with convolutional layers:

convolutional network

A Convolutional Network (LeNet) in 2022



[LeCun et al., 1998]

Normalization

Batch Normalization (BN)

"Batch norm is a widely adopted technique that enables faster and more stable training of deep neural networks"

Consider a batch of activations at some layer. To make each dimension zero-mean unit-variance, apply:

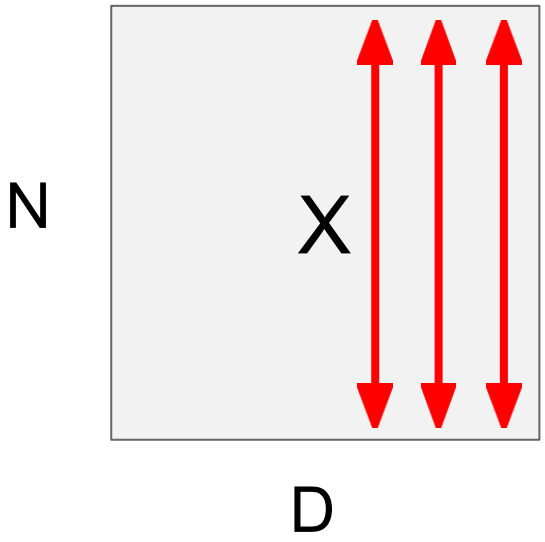
$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

this is a differentiable function...

[Ioffe and Szegedy, 2015]

1D Batch Normalization

1D Case: N*D*1*1 (N = #samples, D = #feat dims)



1. Compute the empirical mean and variance independently for each dimension.

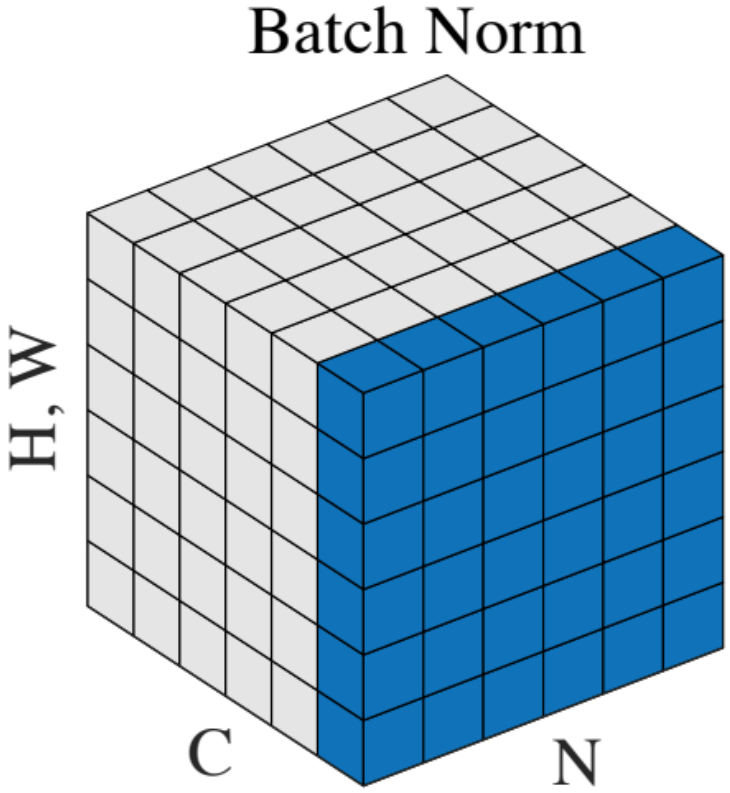
2. Normalize

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

2D Batch Normalization

2D Case: N*D*H*W (HxW = Spatial resolution)

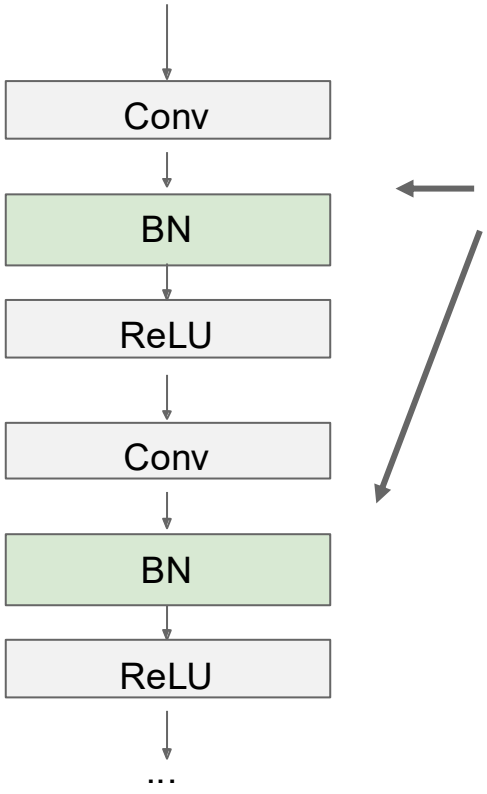
- Spatial Batchnorm: average across HxW



1. Compute the empirical mean and variance independently for each dimension and across all spatial positions

2. Normalize

BN in neural networks



Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

Need a sufficiently large batch size (to aggregate stats within mini-batch)

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

BN: Forward propagation

Normalize:

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

And then allow the network to squash the range if it wants to:

$$y^{(k)} = \gamma^{(k)}\widehat{x}^{(k)} + \beta^{(k)}$$

Note, the network can learn:

$$\gamma^{(k)} = \sqrt{\mathrm{Var}[x^{(k)}]}$$

$$\beta^{(k)} = \mathrm{E}[x^{(k)}]$$

to recover the identity mapping.

BN: Backpropagation

Back propagation

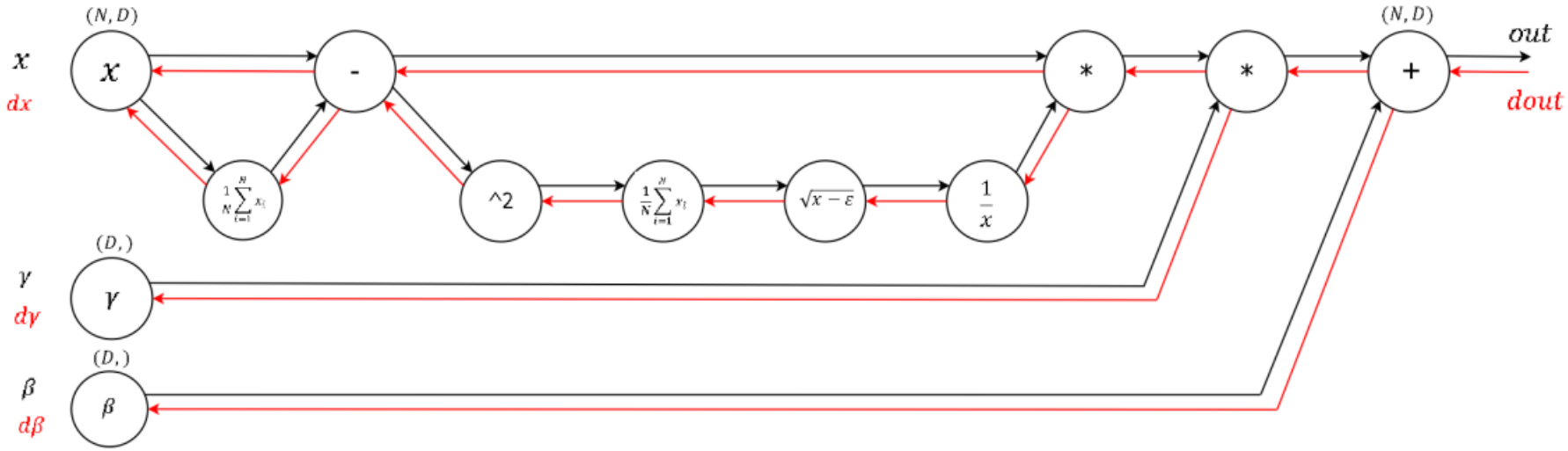


Image source: https://kratzert.github.io/2016/02/12/understanding-the-gradient-flow-through-the-batch-normalization-layer.html

[Ioffe and Szegedy, 2015]

BN: Training and inference

Training

- Take a batch, compute mean and variance
- Use the batch mean and variance to compute new values
- Track “global” mean and variance (e.g., mean = alpha*mean + (1- alpha)*batch_mean)
- Use the new values as input to the next operator

Inference

- Compute “global” mean and variance during training
- Use the "global” mean and variance at test time

BN: Summary

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma\widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

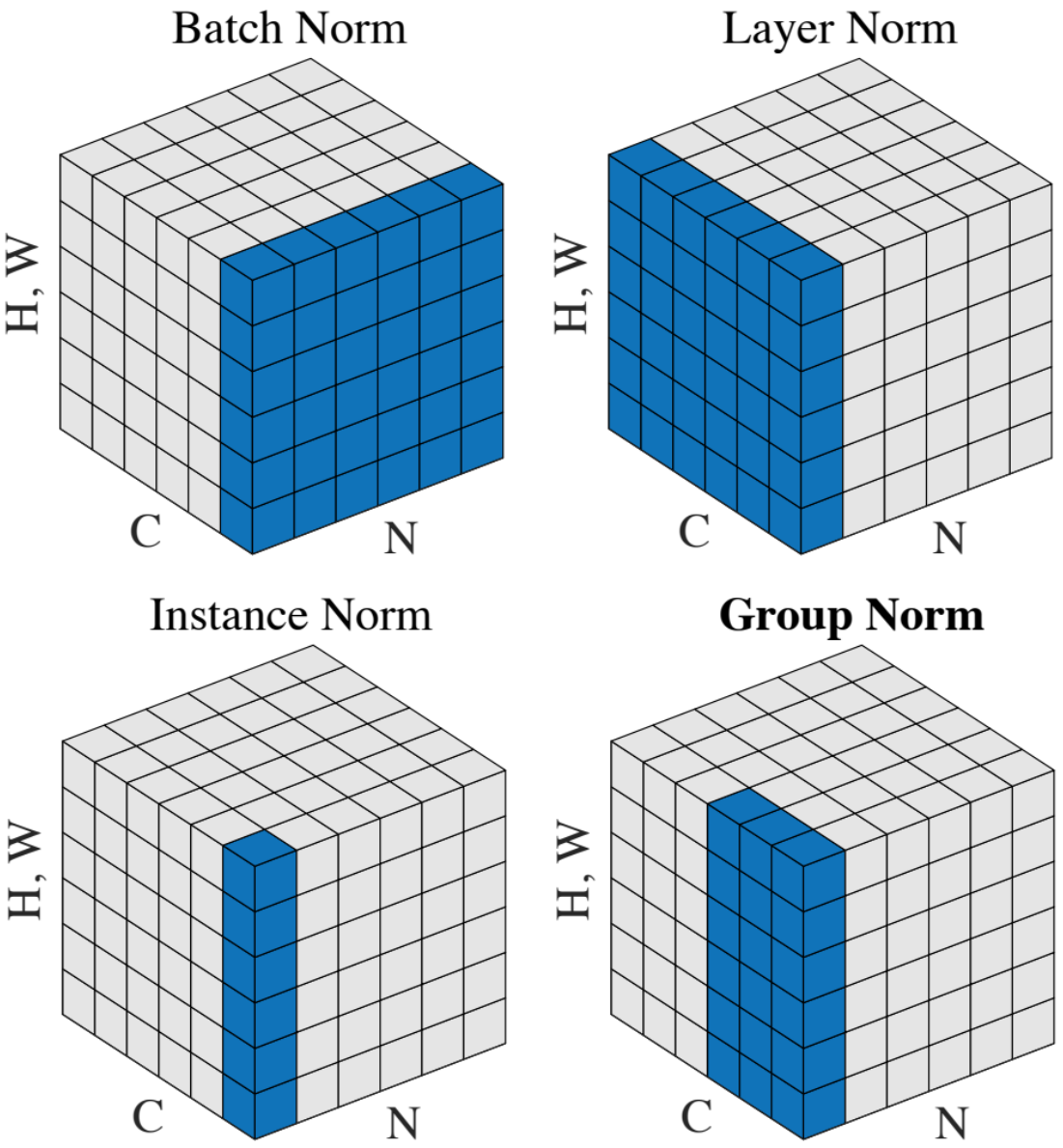
- Improves gradient flow through the network
- Allows higher learning rates
- Reduces the strong dependence on initialization
- Acts as a form of regularization, and slightly reduces the need for dropout, maybe

Exact reasons for BatchNorm's effectiveness are poorly understood ☹

Internal covariate shift? Smooth the optimization landscape?

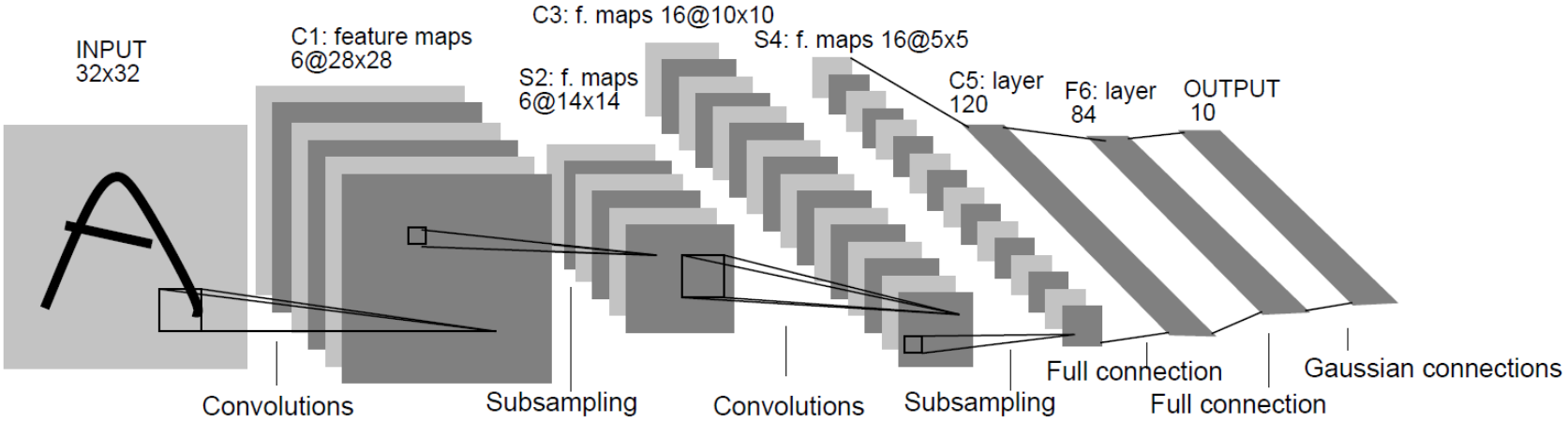
[Ioffe and Szegedy, 2015], [Santurkar, Tsipras, Ilyas, Madry, 2018]

Beyond BN



[Wu and He, 2018]

A Convolutional Network (LeNet) in 2022



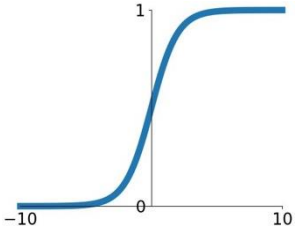
[LeCun et al., 1998]

Nonlinear Activation

Nonlinear Activation Functions

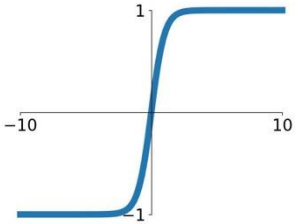
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



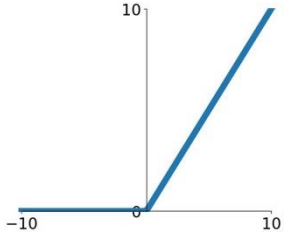
tanh

$$\tanh(x)$$



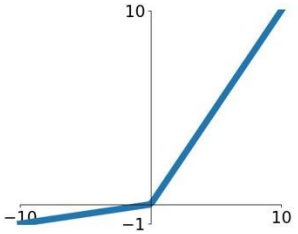
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

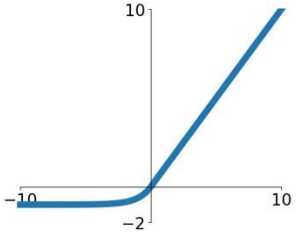


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

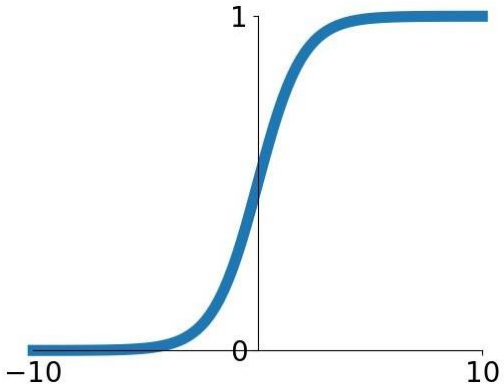
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



All these functions are “differentiable”

Nonlinear Activation Functions

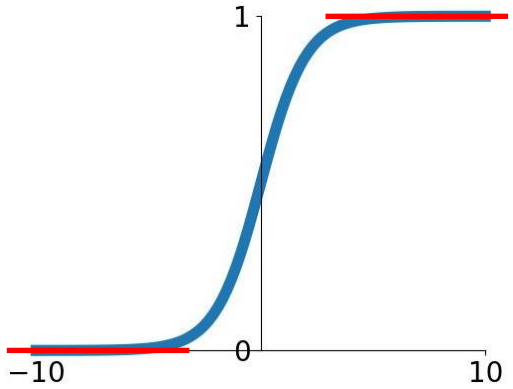


Sigmoid

$$\sigma(x) = 1/(1 + e^{-x})$$

- Squashes numbers to range (0,1)
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron
- Also used for output normalization

Nonlinear Activation Functions



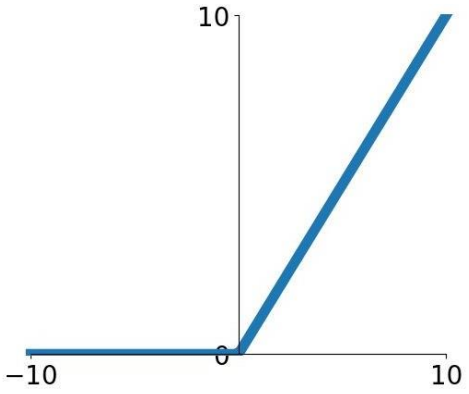
Sigmoid

$$\sigma(x) = 1/(1 + e^{-x})$$

- Squashes numbers to range (0,1)
- Historically popular since they have nice interpretation as a saturating "firing rate" of a neuron
- Also used for output normalization

- Saturated neurons "kill" the gradients
- Exponential function is expensive

Nonlinear Activation Functions



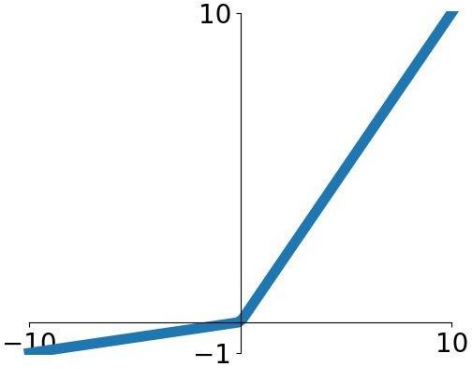
ReLU
(Rectified Linear Unit)

f(x) = max(0, x)

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid in practice (e.g., 6x)
- Actually more biologically plausible than sigmoid

- Gradient knockout (never activate -> never update)

Nonlinear Activation Functions



- Does not saturate (in +region)
- Computationally efficient
- Converges much faster than sigmoid in practice (e.g., 6x)
- Will no “die”

Leaky ReLU

$$f(x) = \max(0.01x, x)$$

Parametric Rectifier (PreLU)

$$f(x) = \max(\alpha x, x)$$

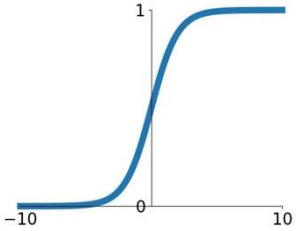
backprop into α (parameter)

[Mass et al., 2013]
[He et al., 2015]

Nonlinear Activation Functions

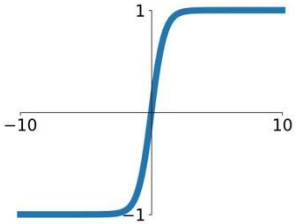
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



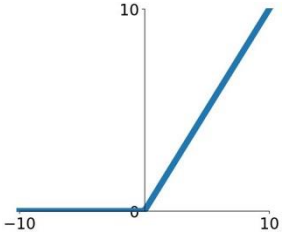
tanh

$$\tanh(x)$$



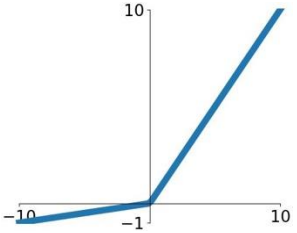
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

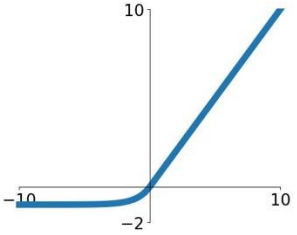


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

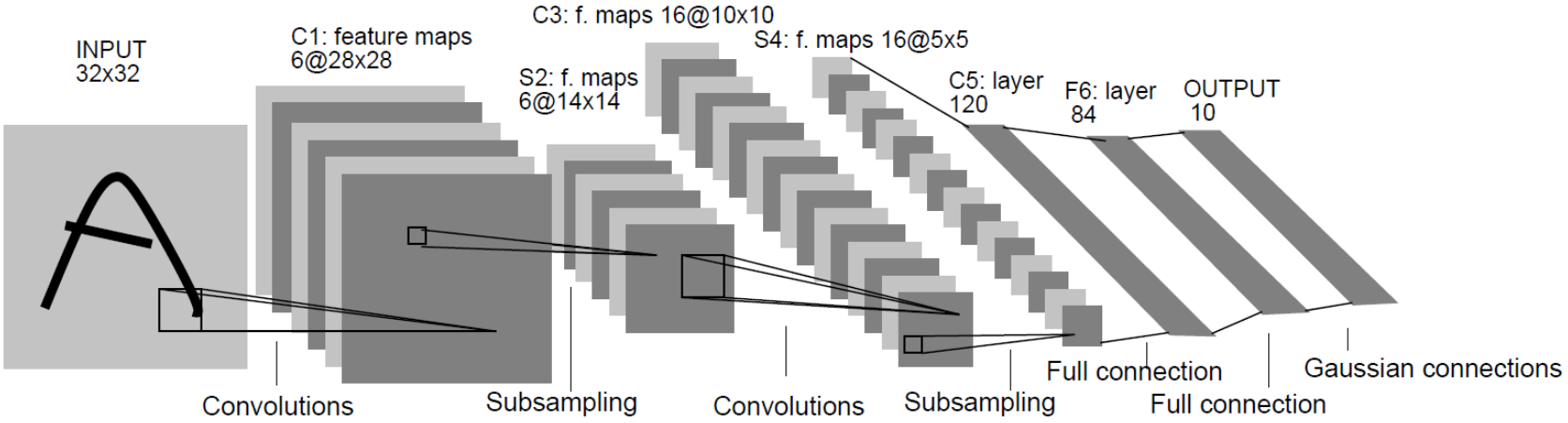
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



ReLU is often used by default

Try out Leaky ReLU, ELU …

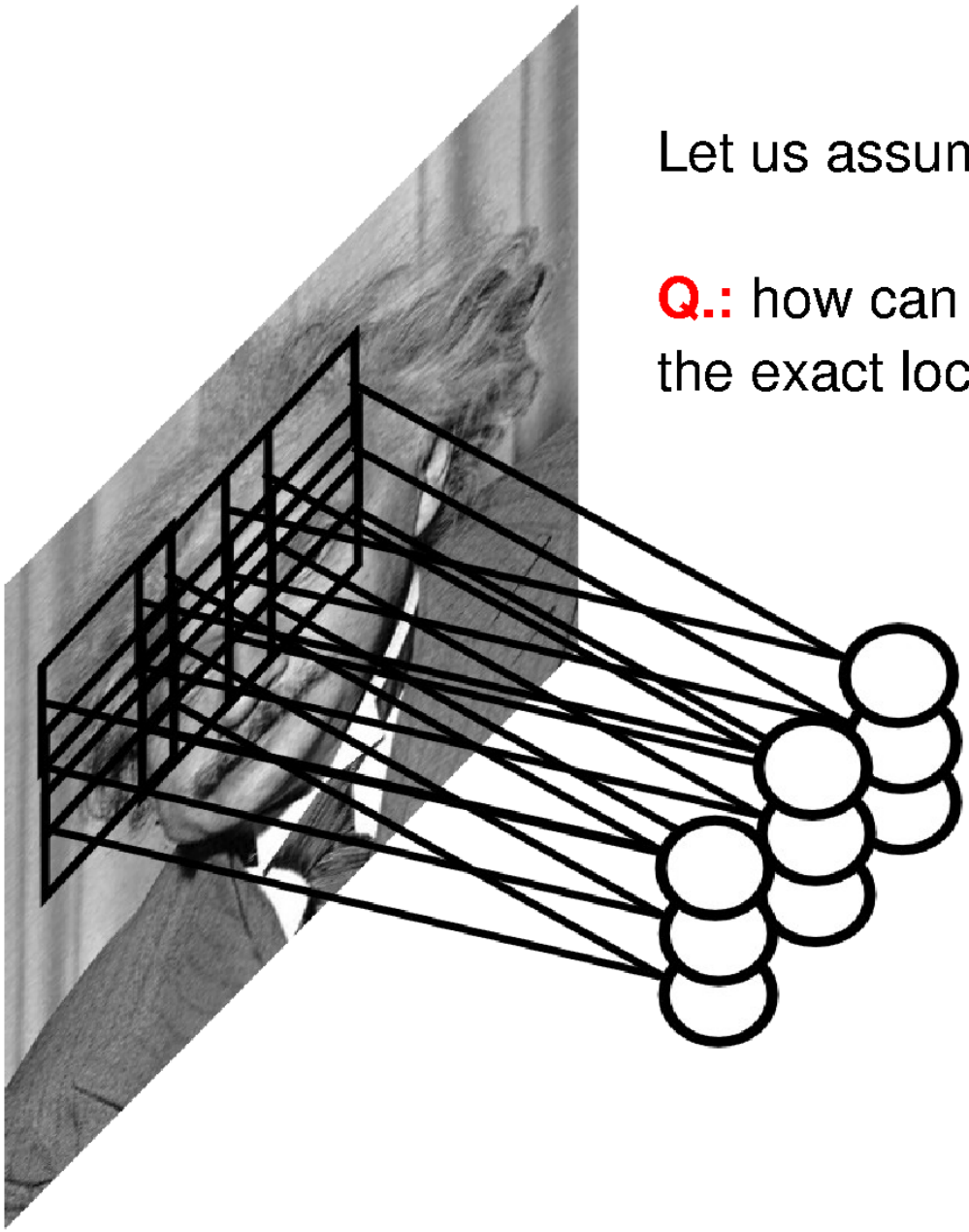
A Convolutional Network (LeNet) in 2022



[LeCun et al., 1998]

Pooling

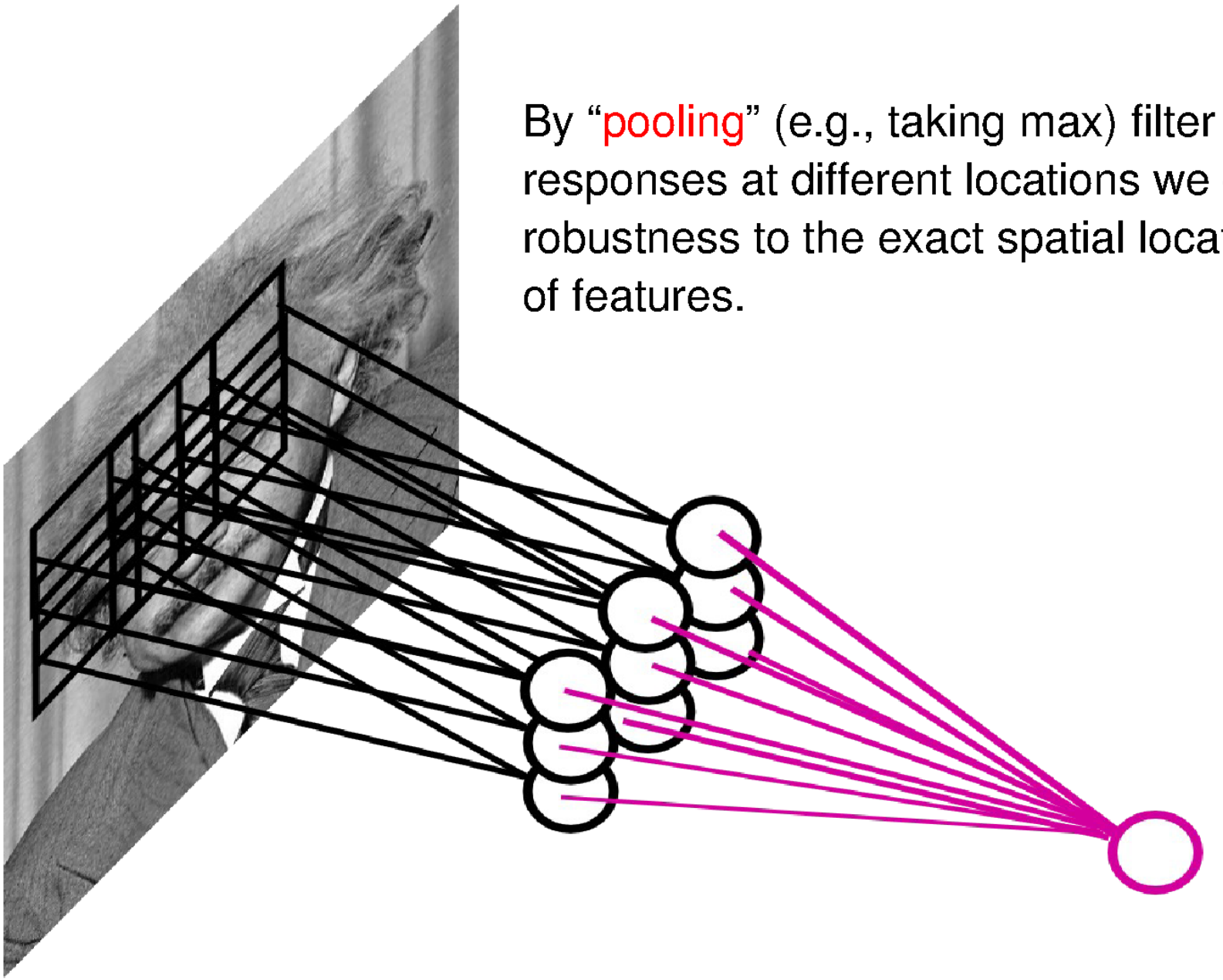
Pooling as a robust detector



Let us assume filter is an “eye” detector.

Q.: how can we make the detection robust to the exact location of the eye?

Pooling as a robust detector



By "pooling" (e.g., taking max) filter responses at different locations we gain robustness to the exact spatial location of features.

Different pooling operations

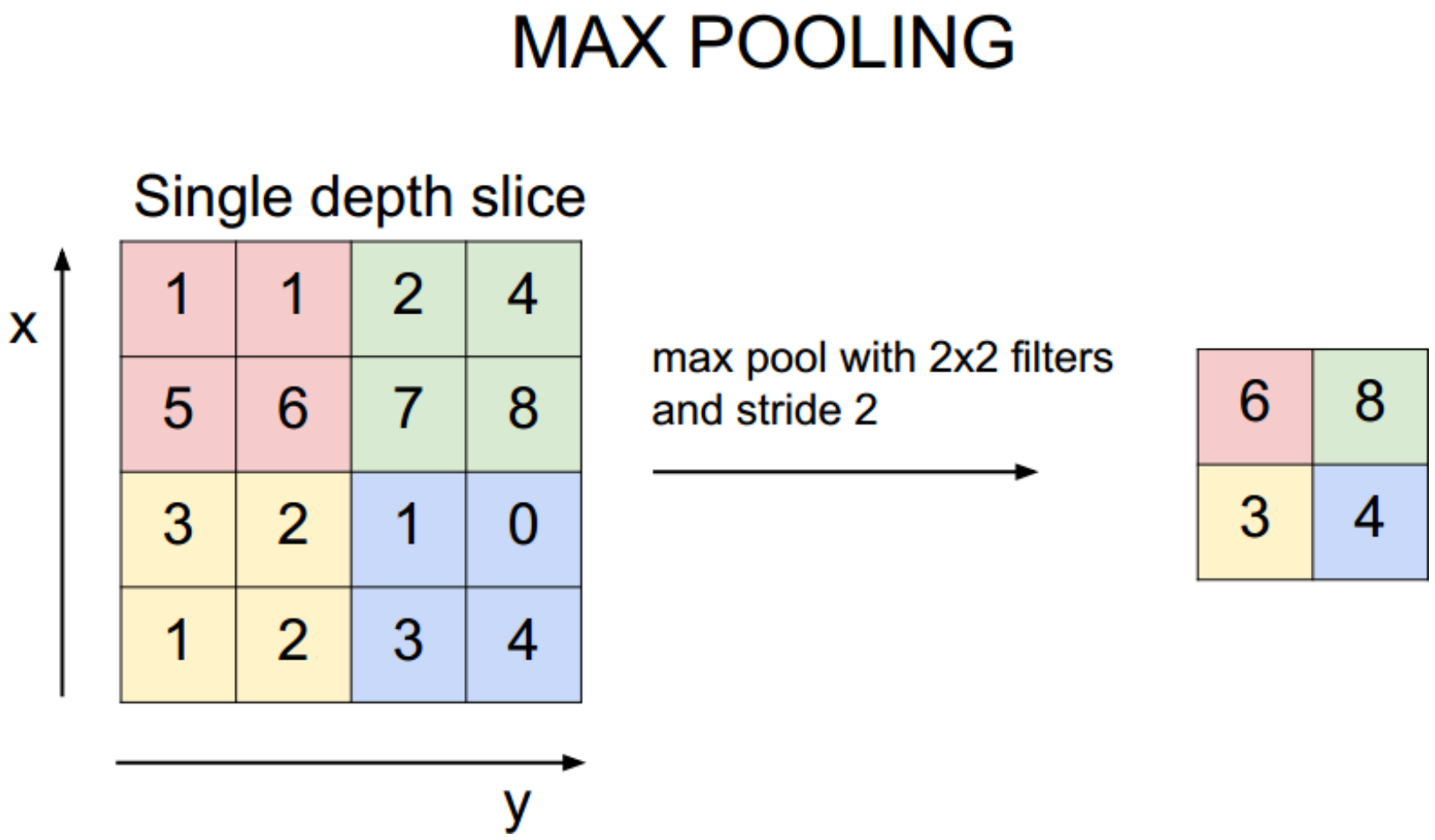
Max-pooling:

$$h_j^n(x, y) = max_{\bar{x} \in N(x),\, \bar{y} \in N(y)} h_j^{n-1}(\bar{x}, \bar{y})$$

Average-pooling:

$$h_j^n(x, y) = 1/K \sum_{\bar{x} \in N(x),\, \bar{y} \in N(y)} h_j^{n-1}(\bar{x}, \bar{y})$$

Example: Max pooling



Question: How to compute the gradients?

Hint: Bookkeeping (save the index in F-Prop)

Pooling operations

Question: What is the size of the output?

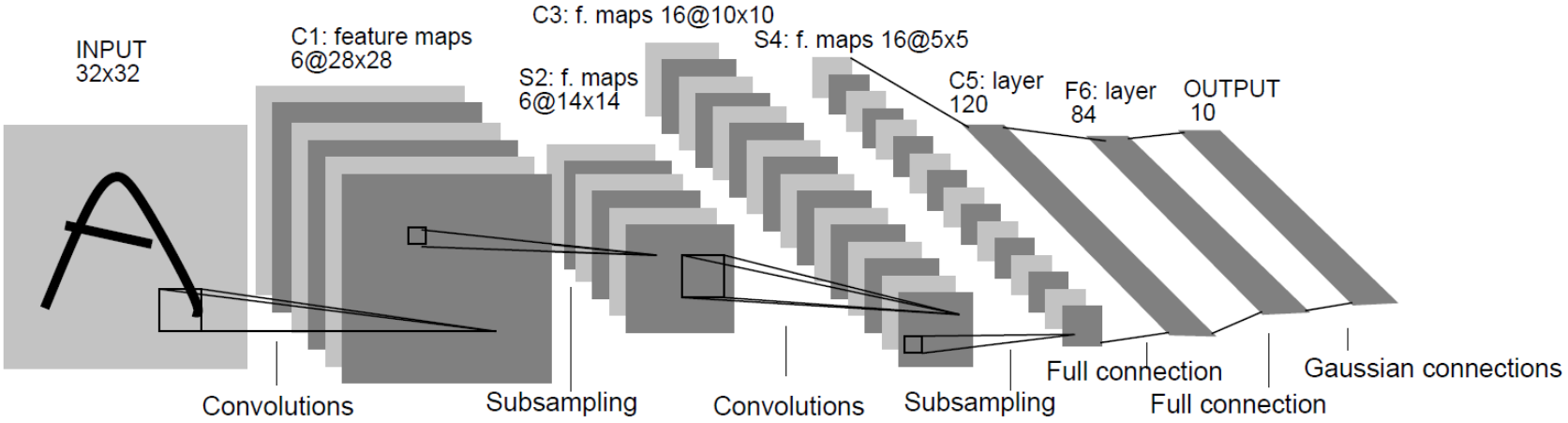
Hint: Think about convolutions!

Question: How should I set the size of the pools?

Answer: It depends on how much “invariant” or robust to distortions we want the representation to be. It is best to pool slowly (via a few stacks of conv-pooling layers).

Make it global: size of the pool = size of the features

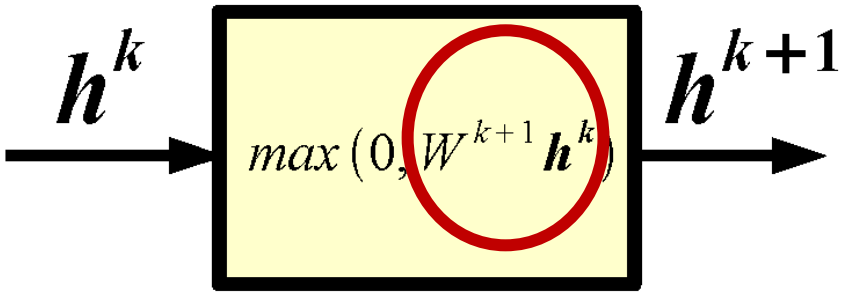
A Convolutional Network (LeNet) in 2022

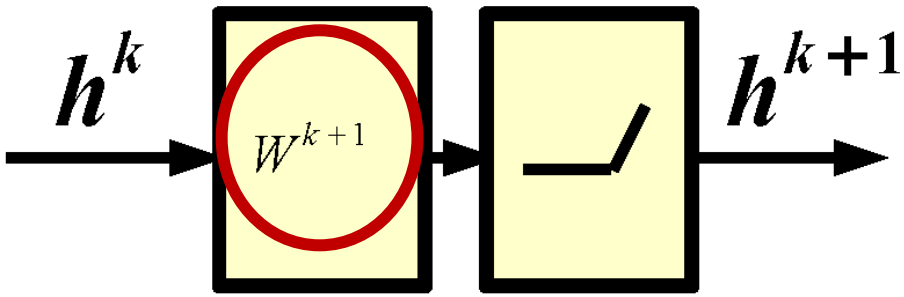


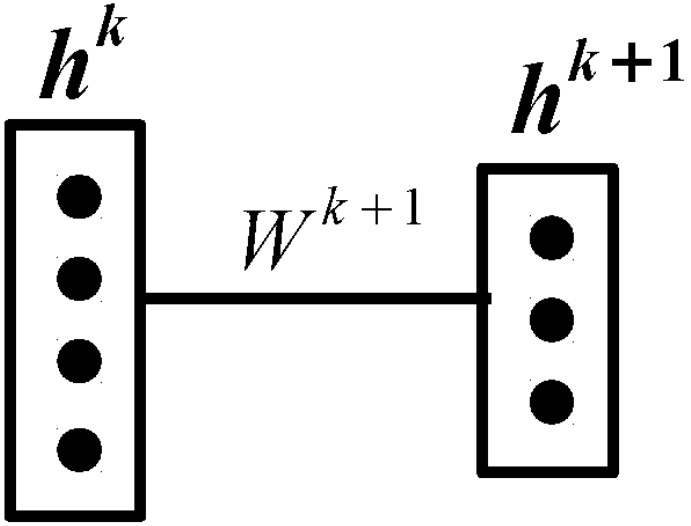
[LeCun et al., 1998]

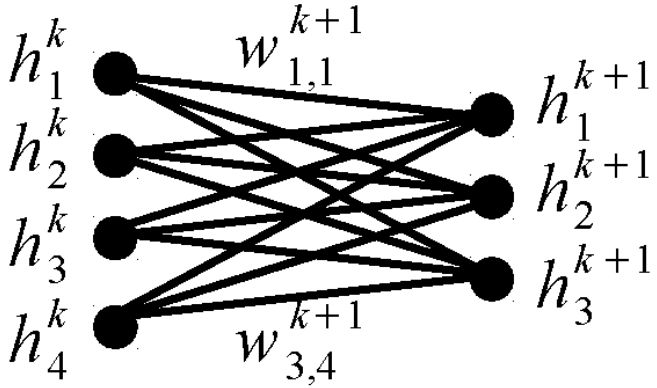
Full Connection

Full connection



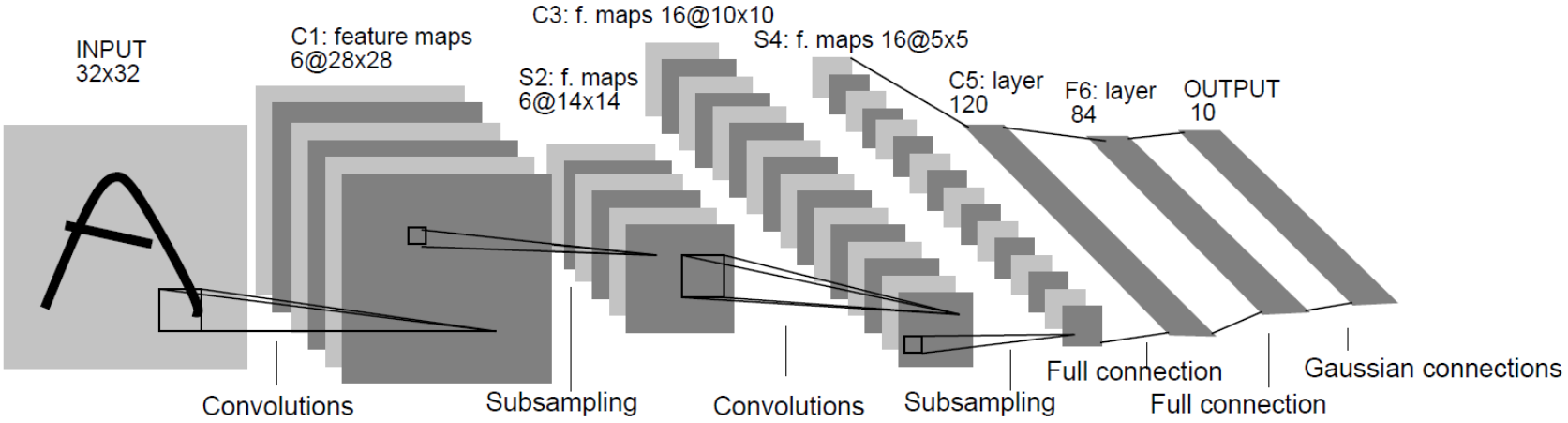






Convolution with kernel size = input size

A Convolutional Network (LeNet) in 2022

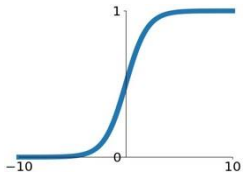


[LeCun et al., 1998]

Output Normalization

Output Normalization

Sigmoid



$$sigmoid(x) = \frac{1}{1 + e^{-x}}$$

- Normalize the outputs into the range of (0,1)
- Can be considered as a probability distribution for a binary random variable
- Does not have any parameters

Output Normalization

Sigmoid

$$sigmoid(x) = \frac{1}{1 + e^{-x}}$$

- Normalize the outputs into the range of [0,1]
- Can be considered as a probability distribution for a binary random variable
- Does not have any parameters

Softmax

$$softmax(x^k) = \frac{\exp(x^k)}{\sum_j \exp(x^j)}$$

- Normalize the outputs into the range of (0,1)
- A probability distribution for a multinomial distribution (*labels are mutually exclusive*)
- Does not have any parameters

Output Normalization

Sigmoid

$$sigmoid(x) = \frac{1}{1 + e^{-x}}$$

- Normalize the outputs into the range of [0,1]
- Can be considered as a probability distribution for a binary random variable
- Does not have any parameters

Softmax

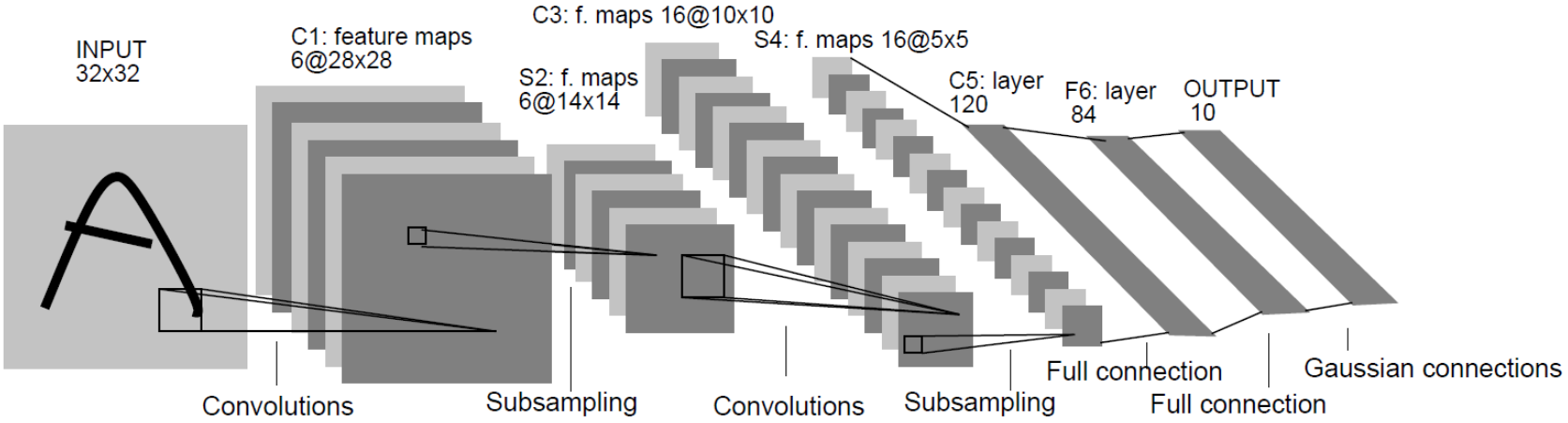
$$softmax(x^k) = \frac{\exp(x^k)}{\sum_j \exp(x^j)}$$

- Normalize the outputs into the range of (0,1)
- A probability distribution for a multinomial distribution (*labels are mutually exclusive*)
- Does not have any parameters

Question: Unstable gradients from the exponential function?

Answer: Merge the normalization into loss function during training

A Convolutional Network (LeNet) in 2022



[LeCun et al., 1998]

Loss Function

Loss Functions

Many choices depending on the task

Cross Entropy Loss $H(y,p) = -\sum_j y_j \log(p_j)$

- Used for classification (assuming mutually exclusive labels)

L1/2 Loss $L_{1/2}(y,\hat{y}) = ||y - \hat{y}||_{1/2}$

- Used for regression

KL Divergence $KL(p,q) = \sum_j p_j \log(\frac{p_j}{q_j})$

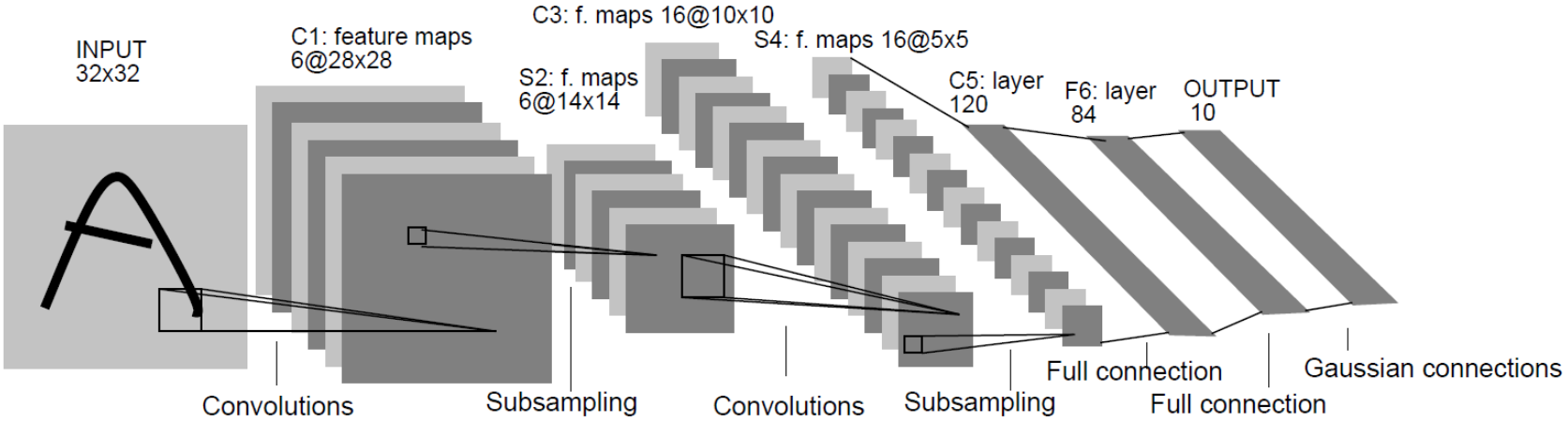
- Matching the output to a reference distribution *q*

Hinge Loss $L_{hinge} = \max(0, 1 - y\hat{y})$

- Approximating the indicator function, used by SVM

Again, all these functions are "differentiable"

A Convolutional Network (LeNet) in 2022



[LeCun et al., 1998]

To Be Continued